

Theoretical Computer Science

Distributional Learning of Context-Free Languages under Fixed Finite-Monoid Typing --Manuscript Draft--

Manuscript Number:	TCS-D-26-00494
Article Type:	Regular Paper (10 - 40 pages)
Section/Category:	Algorithms, automata, complexity and games
Keywords:	
Corresponding Author:	Takayuki Kuriyama Independent Researcher
First Author:	Takayuki Kuriyama
Order of Authors:	Takayuki Kuriyama
Abstract:	We study distributional learning of context-free languages under a fixed recognizable congruence given as the kernel of an explicit finite monoid homomorphism. For this fixed-h setting, we develop a finite typed reconstruction theory for context-free substitutable languages. Starting from a reduced context-free grammar, we introduce a typed refinement that records both yield types and outer context types. We show that the relevant structure is concentrated in a finite typed reconstruction basis and that this basis is exposed by a finite observation set. Thus occurrences of the same nonterminal symbol may have to be separated when their outer h-contexts differ. We then prove exact reconstruction from positive data. From any finite sample, we construct a canonical hypothesis grammar and show that, once the sample contains the finite observation set associated with the target typed grammar, the hypothesis generates the target language exactly. Consequently, for every explicit finite monoid homomorphism h, the class of context-free h-substitutable languages is identifiable in the limit from positive data, with polynomial-time hypothesis construction and update. For the linear subclass, we further prove polynomial upper bounds on characteristic-sample size and word length. Thus the same learner gives a full polynomial time-and-data result for linear targets.

Distributional Learning of Context-Free Languages under Fixed Finite-Monoid Typing

Takayuki Kuriyama
Independent Researcher
growup.kuriyama@gmail.com

April 26, 2026

Abstract

We study distributional learning of context-free languages under a fixed recognizable congruence $\sim_h$ given as the kernel of an explicit finite monoid homomorphism $h : \Sigma^* \rightarrow M$. For this fixed- h setting, we develop a finite typed reconstruction theory for context-free $\sim_h$ -substitutable languages. Starting from a reduced context-free grammar, we introduce a typed refinement that records both yield types and outer context types, show that the relevant structure is concentrated in a finite typed reconstruction basis, and prove that this basis is exposed by a finite observation set. Occurrences of the same nonterminal symbol may therefore have to be separated when their outer h -contexts differ.

We then prove exact reconstruction from positive data. From any finite sample $K \subseteq \Sigma^*$, we construct a canonical hypothesis grammar $\hat{G}(K)$, and we show that once K contains the finite observation set associated with the target typed grammar, $\hat{G}(K)$ generates the target language exactly. Consequently, for every explicit finite monoid homomorphism h , the class $\mathcal{C}_h^{\text{cf}}$ of context-free $\sim_h$ -substitutable languages is identifiable in the limit from positive data, with polynomial-time hypothesis construction and update.

For the linear subclass $\mathcal{C}_h^{\text{lin}}$, we further prove polynomial upper bounds on characteristic-sample size and word length. Thus the same learner gives a full polynomial time-and-data result for linear targets.

keywords: grammatical inference, distributional learning, context-free languages, recognizable congruences, finite monoid homomorphisms, identification in the limit, exact reconstruction

1 Introduction

1.1 Background and motivation

Gold [2] showed that no language class containing all finite languages and at least one infinite language is identifiable in the limit from positive data alone. This negative result motivated a broad line of research in grammatical inference; see, for example, de la Higuera [4] and Heinz, de la Higuera, and van Zaanen [6] for general background. For context-free languages, one standard way to recover positive learning results is the *distributional* approach, in which learnability is obtained by restricting which substrings may be compared across contexts. Classical examples include Clark–Eyraud substitutability [8] and Yoshinaka’s (k, ℓ) -substitutability [9]. More generally, positive-data learnability has also been established for other grammar classes under suitable finite control assumptions; see, for example, Kanazawa [1] for restricted categorial grammars.

The present paper studies a recognizable-congruence version of this distributional program. Let $h : \Sigma^* \rightarrow M$ be a homomorphism into a finite monoid, and write $\sim_h := \ker h$. A language $L \subseteq \Sigma^*$ is $\sim_h$ -substitutable if $h(x) = h(y)$ and $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset$ imply $\mathcal{D}_L(x) = \mathcal{D}_L(y)$ for

all $x, y \in \Sigma^*$. Thus full equality of distributions is required only for strings that already agree under a fixed finite algebraic summary.

Recognizable congruences are a natural comparison mechanism here. If $L = h^{-1}(F)$ for some finite monoid homomorphism $h : \Sigma^* \rightarrow M$ and some $F \subseteq M$, then any two strings with the same h -value automatically have the same continuation behavior, and hence the same distribution. In particular, every regular language belongs to the broader class RS of languages that are $\sim_h$ -substitutable for *some* finite monoid homomorphism. This observation shows that finite monoid typings are not an ad hoc generalization of bounded contexts, but a canonical algebraic source of finite distributional control.

Our focus is the *fixed- h* regime: an explicit finite monoid homomorphism $h : \Sigma^* \rightarrow M$ is given in advance, and the learner is required to learn under that fixed comparison mechanism. In other words, we separate two problems: determining from data which finite congruence should govern comparison, and learning once that congruence has already been specified. This separation is natural. Clark–Eyraud substitutability and Yoshinaka’s (k, ℓ) -substitutability also fix in advance a finite comparison mechanism; the present framework replaces bounded context windows by an arbitrary recognizable congruence.

Relation to prior learners. At the level of surface rule schemata, the learner constructed here still belongs to the distributional tradition: it builds a grammar from local factorizations $uxv \in K$ extracted from a finite positive sample K . The novelty is not this surface format, but the fixed- h structural theory behind it. An explicit homomorphism $h : \Sigma^* \rightarrow M$ induces a typed refinement, a finite reconstruction basis, and a finite observation set sufficient for exact recovery. Clark–Eyraud substitutability appears as the trivial-monoid case. In the genuinely nontrivial fixed- h setting, by contrast, one must simultaneously control yield types and external context types, and the correctness proof proceeds via typed transport and typed substitution rather than the untyped Clark–Eyraud argument.

1.2 Contributions and main results

We do not address the fully adaptive problem of inferring a suitable congruence from positive data alone. Instead, we isolate the learning problem that remains once an explicit finite monoid homomorphism $h : \Sigma^* \rightarrow M$ has been fixed. The resulting theory has two distinct layers. For the general context-free class $\mathcal{C}_h^{\text{cf}}$, we prove exact reconstruction, identification in the limit from positive data, and polynomial-time hypothesis construction and update, but we do *not* claim a polynomial upper bound on the size of a characteristic sample. For the linear subclass $\mathcal{C}_h^{\text{lin}}$, we additionally prove polynomial upper bounds on characteristic-sample size and word length, yielding a full polynomial time-and-data result.

More precisely, the contributions of the paper are as follows.

- (1) We introduce a *typed refinement* $\tilde{G}$ of a reduced context-free grammar under the fixed homomorphism $h : \Sigma^* \rightarrow M$. This refinement separates occurrences of the same untyped nonterminal according to both the h -type of the generated yield and the h -types of the surrounding left and right contexts.
- (2) We define a *finite typed reconstruction basis* $\mathcal{B}(\tilde{G})$ consisting of the reachable productive typed nonterminals, the realized typed rule instances, and the associated canonical terminal and context data. This basis isolates the finite typed local information needed for exact recovery of the target language.
- (3) We show that the typed reconstruction basis is *exposed by a finite observation set* $\text{CS}(\tilde{G})$. Thus the hidden typed structure is not merely postulated abstractly: it is forced by finitely many observable words of the target language.

- (4) From a finite positive sample K , we construct a *canonical hypothesis grammar* $\hat{G}(K)$ and prove an *exact reconstruction theorem*: once K contains $\text{CS}(\hat{G})$, the grammar $\hat{G}(K)$ generates the target language exactly.
- (5) As a learning-theoretic consequence, for every explicit finite monoid homomorphism $h : \Sigma^* \rightarrow M$, the class $\mathcal{C}_h^{\text{cf}}$ is *identifiable in the limit from positive data*. Moreover, for every finite sample, the current hypothesis can be constructed and updated in polynomial time in the current sample size and total input length.
- (6) For the linear subclass $\mathcal{C}_h^{\text{lin}}$, we prove polynomial upper bounds on the size of a characteristic sample and on the lengths of its words. Hence the same learner achieves a full *polynomial time-and-data* result on linear targets.

We further situate the fixed- h framework within the broader landscape of distributional learning. Writing KL for the union of Yoshinaka’s fixed (k, ℓ) -substitutable classes, we show that the capped counter family CCL_p already yields the strict inclusion $\text{KL} \subsetneq \text{RS}$ at the regular level. By contrast, several natural deterministic context-free languages lie outside RS ; examples include the uncapped counter language CCL , the one-bracket Dyck language D_1 , and Yoshinaka’s classical language $L(S \rightarrow aSS \mid b)$.

We also give a positive answer to the question of whether $\text{RS} \setminus \text{KL}$ contains any non-regular context-free language. Concretely, we show that $L^* = \{a^n b^n : n \geq 0\}^*$ is a non-regular deterministic context-free language in $\text{RS} \setminus \text{KL}$, thereby establishing that the strict inclusion $\text{KL} \subsetneq \text{RS}$ extends beyond the regular level to the non-regular context-free level. These boundary results are secondary to the main positive results, but they clarify the scope of recognizable-congruence control. From this perspective, a structural characterization of $\text{RS} \cap \text{CFL}$ remains a natural problem for future work.

Taken together, these results show that the fixed- h setting carves out a mathematically robust and structurally transparent subtheory within distributional context-free learning.

1.3 Organization of the paper

Section 2 recalls the basic notions and positions the classical substitutability frameworks inside the recognizable-congruence setting. Section 3 formulates the fixed- h model and states the main theorem. Section 4 develops the typed refinement and the finite typed reconstruction basis, and Section 5 proves exact reconstruction and identification in the limit. Section 6 establishes polynomial-time upper bounds for hypothesis construction and update, while Section 7 proves polynomial time and data for the linear subclass. Section 8 and Section 9 provide the structural separation and boundary results. Section 10 resolves the open question left by those boundary results by showing that $L^* = \{a^n b^n : n \geq 0\}^*$ is a non-regular deterministic context-free language in $\text{RS} \setminus \text{KL}$, establishing that the strict inclusion $\text{KL} \subsetneq \text{RS}$ extends beyond the regular level. Appendix A records additional closure properties and counterexamples, and Appendix B treats the exceptional case of CCL_1 .

2 Preliminaries

Let Σ^* denote the free monoid over a finite alphabet Σ , and let λ denote its identity element. For a word $w \in \Sigma^*$, we write $|w|$ for its length. For a language $L \subseteq \Sigma^*$ and $x \in \Sigma^*$, the *distribution* of x in L is defined by $\mathcal{D}_L(x) := \{(u, v) \in \Sigma^* \times \Sigma^* : uxv \in L\}$. We use standard asymptotic notation, and all claims about polynomial time are understood with respect to the natural finite input size specified locally at the relevant point.

2.1 Learning from positive data

Throughout the paper, we work in Gold's model of identification in the limit [2]. A *text* for a language $L \subseteq \Sigma^*$ is an infinite sequence $T = (w_1, w_2, \dots)$ of words from L such that every word in L appears at least once in T . For $n \geq 1$, we write $T[n] := (w_1, \dots, w_n)$ for the prefix of T of length n .

A *learner* is a computable mapping that assigns to each finite text prefix a hypothesis grammar, or equivalently the language generated by that grammar. A learner $\mathcal{A}$ *identifies in the limit from positive data* a language class $\mathcal{C}$ if for every $L \in \mathcal{C}$ and every text T for L , there exists n_0 such that for all $n \geq n_0$, we have $L(\mathcal{A}(T[n])) = L$.

Since the learners considered in this paper produce their output independently of the order in which positive examples are observed, we freely identify a finite text prefix $(w_1, \dots, w_n)$ with the corresponding finite sample $K = \{w_1, \dots, w_n\} \subseteq \Sigma^*$.

2.2 Characteristic samples

We adopt the standard characteristic-sample viewpoint in grammatical inference [5, 4]. Let $\mathcal{C}$ be a language class and $\mathcal{A}$ a learner.

Definition 2.1 (Characteristic sample). *Let $L \in \mathcal{C}$. A finite set $S \subseteq L$ is called a characteristic sample of L with respect to $\mathcal{A}$ if $L(\mathcal{A}(K)) = L$ holds for every finite sample K satisfying $S \subseteq K \subseteq L$.*

Definition 2.2 (Polynomial time-and-data, in the standard sense of de la Higuera). *Let $\mathcal{R}$ be a representation class for a language class $\mathcal{C}$, and let $L(r)$ denote the language represented by $r \in \mathcal{R}$. We say that a learner $\mathcal{A}$ identifies $\mathcal{R}$ from positive data in polynomial time and data if there exist polynomials $p_{\mathcal{A}}$ and $q_{\mathcal{A}}$ such that the following hold:*

- (i) *for every finite positive sample K , the hypothesis $\mathcal{A}(K)$ is computable in time at most $p_{\mathcal{A}}(\|K\|)$, where $\|K\| := \sum_{w \in K} |w|$;*
- (ii) *for every target representation $r \in \mathcal{R}$, there exists a finite set $\text{CS}(r) \subseteq L(r)$ such that $\|\text{CS}(r)\| \leq q_{\mathcal{A}}(|r|)$, and for every finite sample K satisfying $\text{CS}(r) \subseteq K \subseteq L(r)$, the hypothesis $\mathcal{A}(K)$ exactly reconstructs $L(r)$.*

2.3 Recognizable congruences and fixed homomorphisms

A congruence on Σ^* is an equivalence relation compatible with concatenation. Such a congruence is called *recognizable* if it has finite index, and this is equivalent to being the kernel of a monoid homomorphism into a finite monoid [3].

Let M be a finite monoid and let $h : \Sigma^* \rightarrow M$ be a monoid homomorphism. We write $\sim_h := \ker h = \{(x, y) \in \Sigma^* \times \Sigma^* : h(x) = h(y)\}$.

Throughout the paper, we assume that h is given *explicitly* to the learner designer: the multiplication table of M and the values $h(a) \in M$ for all $a \in \Sigma$ are given as fixed prior data. Since M is fixed in the main theorem, the time required to compute $h(w)$ for $w \in \Sigma^*$ is linear in $|w|$.

Definition 2.3 ($\sim_h$ -substitutability). *A language $L \subseteq \Sigma^*$ is said to be $\sim_h$ -substitutable if for all $x, y \in \Sigma^*$,*

$$h(x) = h(y) \text{ and } \mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset \implies \mathcal{D}_L(x) = \mathcal{D}_L(y)$$

holds.

As noted above, we write $\mathcal{C}_h^{\text{cf}}$ for the class of context-free $\sim_h$ -substitutable languages, and $\mathcal{C}_h^{\text{lin}}$ for its linear subclass.

3 The Framework of Fixed Finite-Monoid Typing

Fix a finite monoid M and a homomorphism $h : \Sigma^* \rightarrow M$. The aim of this section is to place the fixed- h setting in its proper structural context. We first explain why recognizable congruences provide a natural finite control on distributions, then show that the classical frameworks of Clark–Eyraud and Yoshinaka arise as special cases, and finally state the main fixed- h theorem used throughout the paper.

3.1 Recognizable congruences as distributional control

Every homomorphism $h : \Sigma^* \rightarrow M$ induces an equivalence relation $\sim_h$ on Σ^* by $x \sim_h y \iff h(x) = h(y)$. Since $h(uxv) = h(u)h(x)h(v)$ holds for all $u, x, v \in \Sigma^*$, the relation $\sim_h$ is a monoid congruence of finite index. Such congruences are precisely the recognizable congruences: conversely, every recognizable congruence arises as the kernel of a homomorphism into a finite monoid [3].

The fixed- h setting used below assumes that the homomorphism is given *explicitly*. Concretely, the multiplication table of M and the values $h(a) \in M$ for all $a \in \Sigma$ are included as part of the prior data. Thus the learner is not required to infer the algebraic typing from examples, but instead receives its finite control mechanism in operational form.

Recall that $L \subseteq \Sigma^*$ is $\sim_h$ -substitutable if $h(x) = h(y)$ and $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset$ together imply $\mathcal{D}_L(x) = \mathcal{D}_L(y)$ for all $x, y \in \Sigma^*$.

In other words, two strings are required to have exactly the same full distribution only when they are already equivalent under the fixed finite algebraic summary given by h and already share at least one common context. This is the basic principle of distributional control in the present paper.

For later reference, define

$$\text{RS} := \{L \subseteq \Sigma^* \mid L \text{ is } \sim_h\text{-substitutable for some finite monoid homomorphism } h\},$$

the class of relation-substitutable languages.

Proposition 3.1. *Let $h : \Sigma^* \rightarrow M$ be a finite monoid homomorphism, and let $F \subseteq M$. If $L = h^{-1}(F)$, then L is $\sim_h$ -substitutable. In particular, every regular language belongs to RS.*

Proof. Let $x, y \in \Sigma^*$ satisfy $h(x) = h(y)$ and $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset$. Take any $(u, v) \in \mathcal{D}_L(x)$. Then $uxv \in L = h^{-1}(F)$, so $h(u)h(x)h(v) \in F$. Since $h(x) = h(y)$, we also have $h(u)h(y)h(v) \in F$, hence $uyv \in L$ and $(u, v) \in \mathcal{D}_L(y)$. Therefore $\mathcal{D}_L(x) \subseteq \mathcal{D}_L(y)$, and the reverse inclusion follows symmetrically. $\square$

3.2 Classical substitutability as special cases

Clark–Eyraud substitutability corresponds to the trivial monoid case, and each fixed (k, ℓ) -substitutable class embeds into the present framework via a canonical finite monoid quotient. We record this for completeness.

Definition 3.2 (Clark–Eyraud substitutability). *A language $L \subseteq \Sigma^*$ is substitutable if for all $x, y \in \Sigma^*$, $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset$ implies $\mathcal{D}_L(x) = \mathcal{D}_L(y)$.*

Definition 3.3 (Yoshinaka’s (k, ℓ) -substitutability). *A language $L \subseteq \Sigma^*$ is (k, ℓ) -substitutable if for all $x_1, y_1, z_1, x_2, y_2, z_2 \in \Sigma^*$, all $v \in \Sigma^k$, and all $u \in \Sigma^\ell$, the conditions $vy_1u, vy_2u \neq \lambda$, $x_1vy_1uz_1 \in L$, $x_1vy_2uz_1 \in L$, and $x_2vy_1uz_2 \in L$ together imply $x_2vy_2uz_2 \in L$.*

In other words, (k, ℓ) -substitutability requires a bounded-boundary replacement principle: if two nonempty substrings y_1 and y_2 are both admissible inside one common boundary pair

$v \in \Sigma^k$, $u \in \Sigma^\ell$, and if y_1 is admissible in another ambient context, then y_2 must also be admissible in that second context¹. We write $\text{KL} := \bigcup_{k,\ell \geq 0} \{L \subseteq \Sigma^* \mid L \text{ is } (k, \ell)\text{-substitutable}\}$.

Proposition 3.4. *If L is Clark–Eyraud substitutable, then there exists a finite monoid homomorphism $h : \Sigma^* \rightarrow M$ such that L is $\sim_h$ -substitutable. In particular, every Clark–Eyraud substitutable language belongs to RS.*

Proof. Take the trivial monoid $M = \{1\}$ and let $h : \Sigma^* \rightarrow M$ be the unique homomorphism. Then $h(x) = h(y)$ holds for all $x, y \in \Sigma^*$, so $\sim_h$ -substitutability reduces exactly to Clark–Eyraud substitutability. $\square$

Proposition 3.5. *For each fixed $k, \ell \geq 0$, there exists a finite monoid homomorphism $h_{k,\ell} : \Sigma^* \rightarrow M_{k,\ell}$ such that every (k, ℓ) -substitutable language is $\sim_{h_{k,\ell}}$ -substitutable. In particular, each fixed (k, ℓ) -substitutable class is contained in RS.*

Proof. Fix $k, \ell \geq 0$, and write $N := \max\{1, k + \ell\}$. Also adopt the convention $\text{pref}_0(w) = \text{suf}_0(w) = \lambda$. Define

$$T := \Sigma^{<N} \sqcup (\Sigma^k \times \Sigma^\ell),$$

and let $\tau : \Sigma^* \rightarrow T$ be given by

$$\tau(w) := \begin{cases} w & \text{if } |w| < N, \\ (\text{pref}_k(w), \text{suf}_\ell(w)) & \text{if } |w| \geq N. \end{cases}$$

Since Σ is finite and k, ℓ are fixed, the set T is finite.

Define a relation $\equiv$ on Σ^* by $x \equiv y \iff \tau(x) = \tau(y)$. We first show that $\equiv$ is a congruence. Suppose that $\tau(x) = \tau(x')$ and $\tau(y) = \tau(y')$. It suffices to prove that $\tau(xy) = \tau(x'y')$.

First consider the case $|xy| < N$. Then $|x|, |y| < N$, so $\tau(x) = x$ and $\tau(y) = y$. Since T is defined as the disjoint union $\Sigma^{<N} \sqcup (\Sigma^k \times \Sigma^\ell)$, the equality $\tau(x) = \tau(x')$ implies $x' = x$, and similarly $\tau(y) = \tau(y')$ implies $y' = y$. Hence $\tau(xy) = \tau(x'y')$.

We may therefore assume that $|xy| \geq N$. In this case $\tau(xy)$ is determined by $(\text{pref}_k(xy), \text{suf}_\ell(xy))$, so it is enough to show that both $\text{pref}_k(xy)$ and $\text{suf}_\ell(xy)$ are determined solely by $\tau(x)$ and $\tau(y)$.

First consider the prefix. If $|x| \geq k$, then $\text{pref}_k(xy) = \text{pref}_k(x)$, which is determined by $\tau(x)$. If $|x| < k$, then $\text{pref}_k(xy) = x \text{pref}_{k-|x|}(y)$. In this case $|x| < k \leq k + \ell \leq N$, so $\tau(x) = x$. Moreover, if $|y| < N$, then $\tau(y) = y$, and hence $\text{pref}_{k-|x|}(y)$ is clearly determined by $\tau(y)$. If $|y| \geq N$, then $\tau(y) = (\text{pref}_k(y), \text{suf}_\ell(y))$, and since $k - |x| \leq k$, the word $\text{pref}_{k-|x|}(y)$ is determined by $\text{pref}_k(y)$, hence by $\tau(y)$. Thus $\text{pref}_k(xy)$ is always determined by $\tau(x)$ and $\tau(y)$.

Next consider the suffix. If $|y| \geq \ell$, then $\text{suf}_\ell(xy) = \text{suf}_\ell(y)$, which is determined by $\tau(y)$. If $|y| < \ell$, then $\text{suf}_\ell(xy) = \text{suf}_{\ell-|y|}(x)y$. In this case $|y| < \ell \leq k + \ell \leq N$, so $\tau(y) = y$. Moreover, if $|x| < N$, then $\tau(x) = x$, so $\text{suf}_{\ell-|y|}(x)$ is determined by $\tau(x)$. If $|x| \geq N$, then $\tau(x) = (\text{pref}_k(x), \text{suf}_\ell(x))$, and since $\ell - |y| \leq \ell$, the word $\text{suf}_{\ell-|y|}(x)$ is determined by $\text{suf}_\ell(x)$, hence by $\tau(x)$. Therefore $\text{suf}_\ell(xy)$ is also determined by $\tau(x)$ and $\tau(y)$.

It follows that $\tau(xy)$ is determined solely by $\tau(x)$ and $\tau(y)$. Hence from $\tau(x) = \tau(x')$ and $\tau(y) = \tau(y')$ we obtain $\tau(xy) = \tau(x'y')$. Thus $\equiv$ is a finite-index congruence. Therefore the quotient $M_{k,\ell} := \Sigma^* / \equiv$ is a finite monoid, and the quotient map $h_{k,\ell} : \Sigma^* \rightarrow M_{k,\ell}$ is a finite monoid homomorphism. By construction, $h_{k,\ell}(s) = h_{k,\ell}(t)$ is equivalent to $\tau(s) = \tau(t)$.

Now let $L \subseteq \Sigma^*$ be (k, ℓ) -substitutable, and suppose that $h_{k,\ell}(s) = h_{k,\ell}(t)$ and $\mathcal{D}_L(s) \cap \mathcal{D}_L(t) \neq \emptyset$. We show that $\mathcal{D}_L(s) = \mathcal{D}_L(t)$.

First consider the case $|s| < N$. Then $\tau(s) = s$, which lies in the first component of the disjoint union T . Since $\tau(s) = \tau(t)$ and T is a disjoint union, it follows that $|t| < N$ and $t = s$. Hence $\mathcal{D}_L(s) = \mathcal{D}_L(t)$ trivially.

¹In particular, Yoshinaka's $(0, 0)$ -substitutability may be written as: if $x, y \neq \lambda$ and $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset$, then $\mathcal{D}_L(x) = \mathcal{D}_L(y)$. Thus the quantification is restricted to nonempty strings, so this condition is weaker than Clark–Eyraud substitutability, which requires $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset \Rightarrow \mathcal{D}_L(x) = \mathcal{D}_L(y)$ for all $x, y \in \Sigma^*$.

We may therefore assume that $|s|, |t| \geq N$. Since $\tau(s) = \tau(t)$, we have $\text{pref}_k(s) = \text{pref}_k(t) =: v$ and $\text{suf}_\ell(s) = \text{suf}_\ell(t) =: u$. Thus we may write $s = vy_1u$ and $t = vy_2u$. Moreover, $|s|, |t| \geq N \geq 1$, so $vy_1u = s \neq \lambda$ and $vy_2u = t \neq \lambda$.

Take $(x_1, z_1) \in \mathcal{D}_L(s) \cap \mathcal{D}_L(t)$. Then $x_1vy_1uz_1 \in L$ and $x_1vy_2uz_1 \in L$. Now let $(x_2, z_2) \in \mathcal{D}_L(s)$ be arbitrary. Then $x_2vy_1uz_2 \in L$. Applying the definition of (k, ℓ) -substitutability to $x_1, y_1, z_1, x_2, y_2, z_2, v, u$, we obtain $x_2vy_2uz_2 \in L$. Hence $(x_2, z_2) \in \mathcal{D}_L(t)$. Therefore $\mathcal{D}_L(s) \subseteq \mathcal{D}_L(t)$. The reverse inclusion follows symmetrically by exchanging s and t . Thus $\mathcal{D}_L(s) = \mathcal{D}_L(t)$.

Therefore L is $\sim_{h_{k,\ell}}$ -substitutable. In particular, the fixed (k, ℓ) -substitutable class is contained in RS. $\square$

Corollary 3.6. *Clark–Eyraud substitutability and each fixed (k, ℓ) -substitutable class are special cases of the framework of fixed recognizable congruences.*

Proof. This follows by combining Proposition 3.4 and Proposition 3.5. $\square$

3.3 The main fixed- h theorem

Throughout the paper, we work under the fixed- h framework: a specific finite monoid homomorphism $h : \Sigma^* \rightarrow M$ is given explicitly as part of the prior data and remains fixed throughout the entire learning process. Equivalently, the learner is given the finite algebraic typing in operational form and is not required to infer it from data.

Fix a finite monoid homomorphism $h : \Sigma^* \rightarrow M$. As recalled above, we write

$$\begin{aligned} \mathcal{C}_h^{\text{cf}} &:= \{L \subseteq \Sigma^* \mid L \text{ is context-free and } \sim_h\text{-substitutable}\}, \\ \mathcal{C}_h^{\text{lin}} &:= \{L \subseteq \Sigma^* \mid L \text{ is linear context-free and } \sim_h\text{-substitutable}\}. \end{aligned}$$

Clearly, $\mathcal{C}_h^{\text{lin}} \subseteq \mathcal{C}_h^{\text{cf}}$. The learnability of these classes is the content of our main theorem below.

Theorem 3.7 (Fixed- h learning theorem). *For every explicit finite monoid homomorphism $h : \Sigma^* \rightarrow M$, there exists a learner $\mathcal{A}_h$ such that: (i) $\mathcal{A}_h$ identifies in the limit from positive data every language in $\mathcal{C}_h^{\text{cf}}$; (ii) for every finite sample, the hypothesis constructed by $\mathcal{A}_h$ can be computed and updated in polynomial time in the current sample size and total input length.*

The proof for fixed h is organized as a finite typed reconstruction theory. Section 4 develops the typed local structure and the associated finite reconstruction data, Section 5 proves complete reconstruction from finite observations and derives identification in the limit, and Section 6 establishes polynomial-time upper bounds for hypothesis construction and update.

Corollary 3.8. *Theorem 3.7 specializes both to Clark–Eyraud substitutable languages and to Yoshinaka’s (k, ℓ) -substitutable languages for each fixed k, ℓ .*

Proof. This follows by combining Theorem 3.7 with Proposition 3.4 and Proposition 3.5. $\square$

Remark 3.9. *Theorem 3.7(ii) does not claim a polynomial upper bound on the size of a characteristic sample. Intuitively, beyond the linear subclass, shortest distinguishing contexts may depend on branching derivations or nested structural dependencies, so there is no a priori reason to expect uniformly small samples. By contrast, when one restricts to the linear subclass $\mathcal{C}_h^{\text{lin}}$, the constrained shape of derivations makes it possible to prove that both the size of a characteristic sample and the lengths of its words are bounded by a polynomial in the grammar size (Theorem 7.9).*

The notation specific to typed reconstruction is summarized at the beginning of Section 4.

4 Finite Typed Reconstruction under a Fixed Recognizable Congruence

Under a fixed recognizable congruence, the relevant syntactic information of a $\sim_h$ -substitutable language can be compressed into finitely many typed local configurations. The goal of this section is to make this finite control explicit and to show that complete recovery of the target language follows by reconstructing this finite typed structure from positive data.

The notation in Table 1 will be used throughout the remainder of this paper.

Notation	Meaning
$G = (V, \Sigma, P, S_0)$	a reduced SSBNF grammar for the target language L (Definition 4.1)
$\tilde{G}$	the trimmed typed refinement of G under the fixed homomorphism h (Section 4.1)
$A_p^{m,n}$	the typed copy of the nonterminal A , with yield h -type p and surrounding-context h -type (m, n)
$\omega(X)$	the lexicographically least terminal yield derivable from the productive symbol X
$\chi(X) = \langle u_X, v_X \rangle$	the lexicographically least context pair satisfying $\tilde{S} \Rightarrow_{\tilde{G}}^* u_X X v_X$
$\text{CS}(\tilde{G})$	a finite observation set exposing the typed reconstruction basis of $\tilde{G}$ (Section 4.3)
$\mathcal{B}(\tilde{G})$	the finite typed reconstruction basis of $\tilde{G}$ (Definition 4.11)
$\hat{V}(K)$	the set of observed nonterminals $[x : u, v]$ extracted from factorizations $uxv \in K$
$\hat{G}(K)$	the canonical hypothesis grammar constructed from the finite sample K (Section 5.1)
$\mathcal{A}_h$	the learner defined by $\mathcal{A}_h(w_1, \dots, w_n) = \hat{G}(\{w_1, \dots, w_n\})$

Table 1: Notation specific to typed reconstruction and learner construction.

This section proceeds in the order typed local control $\rightarrow$ finite typed basis. We first introduce the typed refinement $\tilde{G}$ of the target grammar and show that there are only finitely many reachable productive typed nonterminals and finitely many realized typed rule instances. We then collect these into the finite typed reconstruction basis $\mathcal{B}(\tilde{G})$ and show that this basis is exposed by a finite observation set $\text{CS}(\tilde{G})$.

Definition 4.1 (start-separated binary normal form). *A context-free grammar $G = (V, \Sigma, P, S_0)$ is in start-separated binary normal form (SSBNF) if the following conditions hold: (i) S_0 does not occur on the right-hand side of any production; (ii) every production with left-hand side S_0 has the form $S_0 \rightarrow A$ for some $A \in V \setminus \{S_0\}$ or $S_0 \rightarrow \lambda$; (iii) every production with left-hand side $A \in V \setminus \{S_0\}$ has the form $A \rightarrow a$ for some $a \in \Sigma$ or $A \rightarrow BC$ for some $B, C \in V \setminus \{S_0\}$.*

Definition 4.2 (reduced grammar). *A context-free grammar is reduced if every nonterminal is reachable from the start symbol and productive.*

Throughout the paper, we work with a reduced SSBNF grammar for the target language. An equivalent reduced SSBNF grammar can be obtained from any context-free grammar by the standard procedures of start-symbol separation, elimination of useless symbols, and binarization.

4.1 Typed refinement

Even when the underlying nonterminal symbol is the same, different occurrences must be treated as different typed copies if the h -type of the generated yield or the h -types of the surrounding left and right contexts differ. In the fixed- h setting, exact reconstruction depends not only on what a nonterminal can generate, but also on where it can appear with respect to the ambient

recognizable congruence. Thus a nonterminal must carry both an internal yield type and an external left–right context type. Example 4.3 illustrates this distinction concretely.

Example 4.3 (why typing is needed). Let $G = (\{S, A, B\}, \{a, b\}, P, S)$ have productions $S \rightarrow AB \mid BA$, $A \rightarrow a$, and $B \rightarrow b$. Then $L(G) = \{ab, ba\}$, but the same nonterminal A appears in two reachable sentential forms with different context h -types. Thus, if we take the monoid $M = (\mathbb{Z}/2\mathbb{Z}, +, 0)$ and define the homomorphism $h : \{a, b\}^* \rightarrow M$ by $h(a) = 0$ and $h(b) = 1$, then the types distinguish the contexts:

sentential form	context of A	$(h(\text{left context}), h(\text{right context}))$
$S \Rightarrow_G AB$	(λ, b)	$(0, 1)$
$S \Rightarrow_G BA$	(b, λ)	$(1, 0)$.

The typing construction therefore separates these two occurrences of A into distinct typed copies, namely $A_0^{0,1}$ and $A_0^{1,0}$, so that each typed nonterminal carries a globally consistent yield type and outer context type.

We now define the typed refinement of the target grammar. Let $L \in \mathcal{C}_h^{\text{cf}}$, and let $G = (V, \Sigma, P, S_0)$ be any reduced SSBNF grammar (see Definition 4.1) such that $L(G) = L$. Write e_M for the identity element of M .

It is useful to distinguish two grammars. We first define the *full typed refinement* of G , denoted

$$\tilde{G}^{\text{full}} = (\tilde{V}^{\text{full}}, \Sigma, \tilde{P}^{\text{full}}, \tilde{S}).$$

Its start symbol is $\tilde{S}$, and its non-start typed nonterminals are the symbols $A_p^{m,n}$, where $A \in V \setminus \{S_0\}$ and $m, n, p \in M$. The intended meaning is that $A_p^{m,n}$ is a copy of A whose generated yield has h -type p and whose surrounding left and right contexts have h -types m and n , respectively. The rules of $\tilde{G}^{\text{full}}$ are as follows.

- If $S_0 \rightarrow A \in P$, then include $\tilde{S} \rightarrow A_p^{e_M, e_M}$ for every $p \in M$.
- If $S_0 \rightarrow \lambda \in P$, then include $\tilde{S} \rightarrow \lambda$.
- If $A \rightarrow a \in P$ and $h(a) = p$, then include $A_p^{m,n} \rightarrow a$ for every $m, n \in M$.
- If $A \rightarrow BC \in P$, then include

$$A_p^{m,n} \rightarrow B_q^{m,rn} C_r^{mq,n}$$

for all $m, n, q, r \in M$ satisfying $qr = p$.

The superscripts in the binary rule express the propagation of outer context types. If an occurrence of A has left context type m and right context type n , and if the left child has yield type q while the right child has yield type r , then the left child sees right context type rn , and the right child sees left context type mq . The condition $qr = p$ records that the parent yield is the concatenation of the two child yields.

The *trimmed typed refinement* of G , denoted

$$\tilde{G} = (\tilde{V}, \Sigma, \tilde{P}, \tilde{S}),$$

is obtained from $\tilde{G}^{\text{full}}$ by deleting all unreachable symbols, all nonproductive symbols, and all rules incident with such symbols. Unless explicitly stated otherwise, $\tilde{G}$ denotes this trimmed refinement.

The distinction between $\tilde{G}^{\text{full}}$ and $\tilde{G}$ is important. The full refinement contains a typed copy $A_p^{m,n}$ for every $A \in V \setminus \{S_0\}$ and every $m, n, p \in M$, regardless of whether that copy can actually occur in a successful derivation. The trimmed refinement keeps only the reachable and productive typed copies and their realized rules. Thus derivations in $\tilde{G}^{\text{full}}$ may mention typed copies that are later removed, whereas all reconstruction data used below are taken only from the trimmed grammar $\tilde{G}$. In particular, the canonical yield $\omega(X)$, the canonical context $\chi(X)$, the observation set $\text{CS}(\tilde{G})$, and the finite reconstruction basis $\mathcal{B}(\tilde{G})$ are defined only for reachable and productive symbols of $\tilde{G}$. This convention avoids making any claim about typed copies that do not exist after trimming.

4.2 Canonical typed data

Fix a total order on Σ , let $\leq_{\text{slex}}$ be the induced shortlex order on Σ^* , and extend it lexicographically to $(\Sigma^*)^2$. For each productive symbol $\alpha \in (\Sigma \cup \tilde{V})^*$ of $\tilde{G}^2$, let $\omega(\alpha)$ denote the shortlex-minimal terminal string satisfying $\alpha \Rightarrow_G^* \omega(\alpha)$. In particular, if $X \rightarrow YZ$, then $\omega(YZ) = \omega(Y)\omega(Z)$, and if $X \rightarrow a$, then $\omega(a) = a$. Also, for each reachable non-start symbol $X \in \tilde{V} \setminus \{\tilde{S}\}$, let $\chi(X) = \langle u_X, v_X \rangle$ denote the shortlex-minimal pair satisfying $\tilde{S} \Rightarrow_G^* u_X X v_X$.

The following typing invariants are immediate from the construction.

Lemma 4.4 (full typing derivability). *In the full typed refinement $\tilde{G}^{\text{full}}$, if $A \Rightarrow_G^* w \in \Sigma^+$ and $p = h(w)$, then for all $m, n \in M$ we have*

$$A_p^{m,n} \Rightarrow_{\tilde{G}^{\text{full}}}^* w.$$

Proof. We argue by induction on the height of the derivation $A \Rightarrow_G^* w$ in the untyped grammar. If the first rule is $A \rightarrow a$, then $w = a$ and $p = h(a)$, so the rule $A_p^{m,n} \rightarrow a$ belongs to $\tilde{G}^{\text{full}}$ for every $m, n \in M$.

If the first rule is $A \rightarrow BC$ and $w = xy$ with $B \Rightarrow_G^* x$ and $C \Rightarrow_G^* y$, put $q := h(x)$ and $r := h(y)$. Then $qr = h(xy) = p$. By the induction hypothesis,

$$B_q^{m,rn} \Rightarrow_{\tilde{G}^{\text{full}}}^* x \quad \text{and} \quad C_r^{mq,n} \Rightarrow_{\tilde{G}^{\text{full}}}^* y.$$

The full typed refinement contains the rule

$$A_p^{m,n} \rightarrow B_q^{m,rn} C_r^{mq,n},$$

and hence $A_p^{m,n} \Rightarrow_{\tilde{G}^{\text{full}}}^* xy = w$. $\square$

Lemma 4.5 (typing invariants). *For the trimmed typed refinement $\tilde{G}$, the following hold: (i) if $A_p^{m,n} \Rightarrow_{\tilde{G}}^* w \in \Sigma^+$, then $h(w) = p$; (ii) if $\tilde{S} \Rightarrow_{\tilde{G}}^* u A_p^{m,n} v$, then $h(u) = m$ and $h(v) = n$; (iii) $L(\tilde{G}) = L(G) = L$.*

Proof. For (i) and (ii), argue by induction on the height of the derivation in the trimmed typed grammar. Since every rule of $\tilde{G}$ is a rule of $\tilde{G}^{\text{full}}$, the local typing equations are exactly those specified in the full refinement.

For (i), the terminal case follows from rules $A_p^{m,n} \rightarrow a$ with $p = h(a)$. In the binary case, if

$$A_p^{m,n} \rightarrow B_q^{m,rn} C_r^{mq,n}$$

and the two children derive strings of h -types q and r , then the whole yield has h -type $qr = p$.

For (ii), the start case has outer context types (e_M, e_M) by construction. In the binary rule above, if the parent occurs with left and right context types (m, n) , then the left child has left context type m and right context type rn , while the right child has left context type mq and right context type n , exactly as prescribed by the rule. The claim follows by induction along the derivation from $\tilde{S}$.

For (iii), every derivation in $\tilde{G}$ erases typed annotations and gives a derivation in G , so $L(\tilde{G}) \subseteq L(G)$.

Conversely, let $w \in L(G)$. If $w = \lambda$, then $S_0 \rightarrow \lambda$ is a rule of G , so $\tilde{S} \rightarrow \lambda$ is a rule of $\tilde{G}^{\text{full}}$. Since this rule is reachable and productive, it remains in the trimmed grammar $\tilde{G}$, and hence $\lambda \in L(\tilde{G})$.

Now suppose $w \neq \lambda$. Choose a derivation $S_0 \Rightarrow_G A \Rightarrow_G^* w$ and put $p := h(w)$. In $\tilde{G}^{\text{full}}$ the rule $\tilde{S} \rightarrow A_p^{e_M, e_M}$ exists, and by Lemma 4.4 we have $A_p^{e_M, e_M} \Rightarrow_{\tilde{G}^{\text{full}}}^* w$. Therefore the symbol $A_p^{e_M, e_M}$ is reachable from $\tilde{S}$ and productive in $\tilde{G}^{\text{full}}$. Moreover, the rules appearing in one such successful derivation from $\tilde{S}$ to w are all incident only with reachable and productive symbols. Hence they survive the trimming step, and the same derivation exists in $\tilde{G}$. Thus $w \in L(\tilde{G})$. $\square$

²That is, a string of nonterminals from which one can eventually derive a terminal string.

Lemma 4.6. *If $X = A_p^{m,n}$ is reachable and productive in $\tilde{G}$, then $h(u_X) = m$, $h(\omega(X)) = p$, and $h(v_X) = n$.*

Proof. Since $\tilde{S} \Rightarrow_{\tilde{G}}^* u_X A_p^{m,n} v_X$, Lemma 4.5(ii) gives $h(u_X) = m$ and $h(v_X) = n$. Since $X \Rightarrow_{\tilde{G}}^* \omega(X)$, Lemma 4.5(i) gives $h(\omega(X)) = p$. $\square$

4.3 Typed local control

We extract the finite local typed information that will later be used for reconstruction. Define the characteristic sample by

$$\text{CS}(\tilde{G}) := \{u_X \omega(\beta) v_X \mid X \rightarrow \beta \in \tilde{P} \text{ and } X \in \text{NT}(\tilde{G}), X \neq \tilde{S}\} \cup (\{\lambda\} \text{ if } \tilde{S} \rightarrow \lambda \in \tilde{P}),$$

where $\text{NT}(\tilde{G})$ denotes the set of reachable and productive typed nonterminals of $\tilde{G}$.

Lemma 4.7 (observation words are positive examples). *We have $\text{CS}(\tilde{G}) \subseteq L(\tilde{G}) = L$.*

Proof. Let $z \in \text{CS}(\tilde{G})$. If $z = \lambda$, then $\tilde{S} \rightarrow \lambda$ is a rule of $\tilde{G}$, and hence $\lambda \in L(\tilde{G})$.

Otherwise, by the definition of $\text{CS}(\tilde{G})$, there is a realizable non-start rule $X \rightarrow \beta$ of $\tilde{G}$ such that $z = u_X \omega(\beta) v_X$. Since X is reachable, the definition of $\chi(X) = \langle u_X, v_X \rangle$ gives

$$\tilde{S} \Rightarrow_{\tilde{G}}^* u_X X v_X.$$

Since $X \rightarrow \beta$ is a rule of the trimmed grammar and its right-hand side is productive, we have

$$\beta \Rightarrow_{\tilde{G}}^* \omega(\beta).$$

Therefore

$$\tilde{S} \Rightarrow_{\tilde{G}}^* u_X X v_X \Rightarrow_{\tilde{G}} u_X \beta v_X \Rightarrow_{\tilde{G}}^* u_X \omega(\beta) v_X = z.$$

Thus $z \in L(\tilde{G})$. By Lemma 4.5(iii), $L(\tilde{G}) = L$, so $z \in L$. $\square$

Lemma 4.8. *If X is reachable and productive in $\tilde{G}$, then $u_X \omega(X) v_X \in \text{CS}(\tilde{G})$.*

Proof. Choose a derivation $X \Rightarrow_{\tilde{G}}^* \omega(X)$, and let $X \rightarrow \beta$ be the first rule used in this derivation. By definition, $\omega(X) = \omega(\beta)$. Hence $u_X \omega(X) v_X = u_X \omega(\beta) v_X \in \text{CS}(\tilde{G})$. $\square$

Theorem 4.9 (Typed local control). *$\text{CS}(\tilde{G})$ is well defined and satisfies $|\text{CS}(\tilde{G})| < \infty$. In other words, all local derivational information relevant to reconstruction is carried by finitely many typed local configurations.*

Proof. The full typed refinement already has the finite nonterminal set

$$\{\tilde{S}\} \cup \{A_p^{m,n} \mid A \in V \setminus \{S_0\}, m, n, p \in M\}.$$

The trimmed grammar $\tilde{G}$ is obtained by deleting symbols and rules from this finite grammar, so it is finite.

For each productive symbol X of $\tilde{G}$, the set $\{w \in \Sigma^* \mid X \Rightarrow_{\tilde{G}}^* w\}$ is nonempty, so the shortlex least element $\omega(X)$ is well defined. Similarly, for each reachable non-start symbol X of $\tilde{G}$, the set

$$\{\langle u, v \rangle \in (\Sigma^*)^2 \mid \tilde{S} \Rightarrow_{\tilde{G}}^* u X v\}$$

is nonempty, so the shortlex least pair $\chi(X) = \langle u_X, v_X \rangle$ is well defined. Since $\tilde{G}$ has only finitely many reachable and productive symbols and rules, the set of observation words $u_X \omega(\beta) v_X$ appearing in the definition of $\text{CS}(\tilde{G})$ is finite. Therefore $\text{CS}(\tilde{G})$ is well defined and finite. $\square$

Remark 4.10. *The point of Theorem 4.9 is that, after typing, the information about the local behavioral conditions that the target grammar must satisfy for reconstruction is compressed into finitely many typed objects.*

4.4 Finite typed reconstruction basis

We now gather the finite typed local information of $\tilde{G}$ into a single reconstruction object.

Definition 4.11 (finite typed reconstruction basis). *Define the finite typed reconstruction basis of $\tilde{G}$ by*

$$\mathcal{B}(\tilde{G}) := (\text{NT}(\tilde{G}), \text{Rule}(\tilde{G}), \omega, \chi, \text{CS}(\tilde{G})).$$

Here, $\text{NT}(\tilde{G})$ is the set of reachable and productive typed nonterminals of $\tilde{G}$, and $\text{Rule}(\tilde{G})$ is the set of rules whose symbols are all reachable and productive typed nonterminals of $\tilde{G}$ (we call these the “realizable rules”).

Theorem 4.12 (existence of a finite typed reconstruction basis). *The basis $\mathcal{B}(\tilde{G})$ is well-defined and finite. Moreover, each reachable and productive typed nonterminal is exhibited by its anchor word $u_X\omega(X)v_X \in \text{CS}(\tilde{G})$, and each realizable rule $X \rightarrow \beta$ is exhibited by its canonical observation word $u_X\omega(\beta)v_X \in \text{CS}(\tilde{G})$.*

Proof. Finiteness follows from Theorem 4.9. By Lemma 4.8, each reachable and productive typed nonterminal X is exhibited by its anchor word $u_X\omega(X)v_X$. Also, by the definition of $\text{CS}(\tilde{G})$, each realized non-start rule $X \rightarrow \beta$ is exhibited by the corresponding canonical observation word $u_X\omega(\beta)v_X$. $\square$

Remark 4.13. *In this notation, the characteristic sample $\text{CS}(\tilde{G})$ is the terminal observation set that exhibits the finite typed reconstruction basis $\mathcal{B}(\tilde{G})$; that is, it serves as an observation window through which the hidden typed structure is made visible. Thus the characteristic sample is by no means merely an auxiliary set added later only for the convergence proof. Rather, it is precisely the working space, or finite observational interface, on which the fixed- h reconstruction theory operates.*

5 Exact Reconstruction and Its Learning-Theoretic Consequences

Let $L \in \mathcal{C}_h^{\text{cf}}$, let $G = (V, \Sigma, P, S_0)$ be a reduced SSBNF grammar for L , and let $\tilde{G}$ be the typed grammar obtained in Section 4. By Theorem 4.12, the finite typed local structure of $\tilde{G}$ is exposed by the finite observation set $\text{CS}(\tilde{G})$. In this section, we show that whenever a finite sample contains that observation set, the canonical learner grammar exactly recovers the target language. Identification in the limit then follows as an immediate consequence.

The proof of exact reconstruction proceeds in two stages. First, Lemmas 5.2, 5.3, and 5.4 establish three local facts: that the typed reconstruction basis is visible in $\hat{G}(K)$ whenever K contains $\text{CS}(\tilde{G})$; that typed nonterminals can be transported between outer contexts; and that internal strings with equal h -type can be substituted within a fixed outer context. Next, Theorem 5.5 combines these facts by induction on derivation height to prove $L \subseteq L(\hat{G}(K))$, and Theorem 5.6 proves $L(\hat{G}(K)) \subseteq L$ by induction on derivation height in $\hat{G}(K)$.

5.1 The learner

For a finite set $K \subseteq \Sigma^*$, define $\hat{V}(K) := \{[x : u, v] \mid uxv \in K\}$, and let $\hat{G}(K) = (\hat{V}(K) \cup \{\hat{S}\}, \Sigma, \hat{P}(K), \hat{S})$ be the grammar with the following rule families.

- (1) If all three nonterminals belong to $\hat{V}(K)$, then $[xy : u, v] \rightarrow [x : u, yv] [y : ux, v]$;
- (2) if $[x : u, v], [x : u', v'] \in \hat{V}(K)$, then $[x : u, v] \rightarrow [x : u', v']$;
- (3) if $[x : u, v], [x' : u, v] \in \hat{V}(K)$ and $h(x) = h(x')$, then $[x : u, v] \rightarrow [x' : u, v]$;
- (4) for $a \in \Sigma$, $[a : u, v] \rightarrow a$;
- (5) for $w \in K$, $\hat{S} \rightarrow [w : \lambda, \lambda]$, and if $\lambda \in K$, then also $\hat{S} \rightarrow \lambda$.

For a finite text prefix $(w_1, \dots, w_n)$, define the learner by $\mathcal{A}_h(w_1, \dots, w_n) := \hat{G}(\{w_1, \dots, w_n\})$.

Remark 5.1. The soundness of Rule (2) is established in the proof of Theorem 5.6 below. Concretely, if $[x : u, v] \rightarrow [x : u', v']$ and then $[x : u', v'] \Rightarrow^* y$, the induction in Theorem 5.6 yields $u' y v' \in L$ and $h(y) = h(x)$, while $u' x v', u x v \in K \subseteq L$ give $(u', v') \in \mathcal{D}_L(x)$ and $(u, v) \in \mathcal{D}_L(x)$. Hence $(u', v') \in \mathcal{D}_L(x) \cap \mathcal{D}_L(y)$, so by $\sim_h$ -substitutability we obtain $\mathcal{D}_L(x) = \mathcal{D}_L(y)$, and therefore $(u, v) \in \mathcal{D}_L(y)$, that is, $u y v \in L$.

Rule (3) performs substitution within a fixed outer context: it replaces the internal fragment by another string with the same h -value while leaving the surrounding context unchanged. A rule $[x : u, v] \rightarrow [x' : u, v]$ exists only when $[x : u, v], [x' : u, v] \in \hat{V}(K)$, that is, only when $u x v, u x' v \in K \subseteq L$. Hence $(u, v) \in \mathcal{D}_L(x) \cap \mathcal{D}_L(x')$. Combining this with $h(x) = h(x')$ and the $\sim_h$ -substitutability of L , we obtain $\mathcal{D}_L(x) = \mathcal{D}_L(x')$.

For the remainder of this section, fix a finite sample K satisfying $\text{CS}(\tilde{G}) \subseteq K \subseteq L$.

5.2 Local realization, transport, and substitution

We isolate the local facts needed to lift derivations of $\tilde{G}$ to derivations of $\hat{G}(K)$.

Lemma 5.2 (Local rule realization). *Let $X \rightarrow \beta$ be a realized non-start rule of $\tilde{G}$. Then its canonical observation word $u_X \omega(\beta) v_X$ belongs to K , and the corresponding local configuration is visible in $\hat{G}(K)$. More precisely, (i) if $X \rightarrow a$, then $[\omega(X) : u_X, v_X] \rightarrow a$ is a rule of $\hat{G}(K)$, and (ii) if $X \rightarrow YZ$, then $[\omega(X) : u_X, v_X], [\omega(Y)\omega(Z) : u_X, v_X], [\omega(Y) : u_X, \omega(Z)v_X]$, and $[\omega(Z) : u_X\omega(Y), v_X]$ all belong to $\hat{V}(K)$.*

Proof. Since $X \rightarrow \beta$ is a realized non-start rule of $\tilde{G}$, its canonical observation word $u_X \omega(\beta) v_X$ belongs by definition to $\text{CS}(\tilde{G})$, and hence to K .

If $X \rightarrow a$, then $\omega(X) = a$, so $u_X a v_X \in K$, and therefore $[a : u_X, v_X] \in \hat{V}(K)$. Hence, by Rule (4), $[\omega(X) : u_X, v_X] \rightarrow a$ is a rule of $\hat{G}(K)$.

Now suppose $X \rightarrow YZ$. Then $u_X \omega(Y)\omega(Z) v_X = u_X \omega(\beta) v_X \in K^3$. Therefore, from the three factorizations $u_X \cdot \omega(Y)\omega(Z) \cdot v_X$, $u_X \cdot \omega(Y) \cdot \omega(Z)v_X$, and $u_X \omega(Y) \cdot \omega(Z) \cdot v_X$, it follows that $[\omega(Y)\omega(Z) : u_X, v_X]$, $[\omega(Y) : u_X, \omega(Z)v_X]$, and $[\omega(Z) : u_X\omega(Y), v_X]$ all belong to $\hat{V}(K)$. Furthermore, by Lemma 4.8, $u_X \omega(X) v_X \in \text{CS}(\tilde{G}) \subseteq K$, and hence $[\omega(X) : u_X, v_X] \in \hat{V}(K)$ as well. $\square$

Lemma 5.3 (Transport). *If $[x : u, v], [x : u', v'] \in \hat{V}(K)$, then $[x : u, v] \Rightarrow_{\hat{G}(K)} [x : u', v']$.*

Proof. This is exactly Rule (2). $\square$

Lemma 5.4 (Substitution). *If $[x : u, v], [x' : u, v] \in \hat{V}(K)$ and $h(x) = h(x')$, then $[x : u, v] \Rightarrow_{\hat{G}(K)} [x' : u, v]$.*

Proof. This is exactly Rule (3). $\square$

5.3 Exact reconstruction

Theorem 5.5. *If K is finite and satisfies $\text{CS}(\tilde{G}) \subseteq K \subseteq L$, then $L \subseteq L(\hat{G}(K))$.*

Proof. We prove the following claim by induction on the height of a fixed derivation $X \Rightarrow_{\tilde{G}}^* w$.

Claim. If X is a reachable and productive non-start symbol of $\tilde{G}$, $\chi(X) = \langle u_X, v_X \rangle$, and $X \Rightarrow_{\tilde{G}}^* w$, then $[\omega(X) : u_X, v_X] \Rightarrow_{\hat{G}(K)}^* w$.

If the derivation height in $\tilde{G}$ is 1, then $X \rightarrow a$ and $\omega(X) = a$. By Lemma 5.2, the rule $[\omega(X) : u_X, v_X] \rightarrow a = w$ belongs to $\hat{G}(K)$.

³Here $\beta = YZ$. Moreover, since ω is defined as the canonical terminal yield of a productive string of symbols, we have $\omega(\beta) = \omega(YZ) = \omega(Y)\omega(Z)$.

Now consider the case where the derivation height is greater than 1. Since G is in SSBNF, the first non-start rule is binary, so let $X \rightarrow YZ$, $Y \Rightarrow_{\tilde{G}}^* w_1$, $Z \Rightarrow_{\tilde{G}}^* w_2$, and $w = w_1w_2$. Since X is reachable and productive, and the fixed derivation $X \Rightarrow_{\tilde{G}}^* w$ begins with $X \rightarrow YZ$ and continues with $Y \Rightarrow_{\tilde{G}}^* w_1$ and $Z \Rightarrow_{\tilde{G}}^* w_2$, both Y and Z are reachable and productive as well. Hence $X \rightarrow YZ$ is a realised non-start rule of $\tilde{G}$. By Lemma 5.2, we have $[\omega(X) : u_X, v_X]$, $[\omega(Y)\omega(Z) : u_X, v_X]$, $[\omega(Y) : u_X, \omega(Z)v_X]$, and $[\omega(Z) : u_X\omega(Y), v_X] \in \hat{V}(K)$, and by Rule (1), $[\omega(Y)\omega(Z) : u_X, v_X] \rightarrow [\omega(Y) : u_X, \omega(Z)v_X][\omega(Z) : u_X\omega(Y), v_X]$. Moreover, by Lemma 4.6, $h(\omega(X)) = h(\omega(Y)\omega(Z))^4$, so by Lemma 5.4, $[\omega(X) : u_X, v_X] \Rightarrow_{\hat{G}(K)} [\omega(Y)\omega(Z) : u_X, v_X]$.

By Lemma 4.8, we have $u_Y\omega(Y)v_Y, u_Z\omega(Z)v_Z \in \text{CS}(\tilde{G}) \subseteq K$, and hence $[\omega(Y) : u_Y, v_Y], [\omega(Z) : u_Z, v_Z] \in \hat{V}(K)$. The source nonterminals $[\omega(Y) : u_X, \omega(Z)v_X]$ and $[\omega(Z) : u_X\omega(Y), v_X]$ belong to $\hat{V}(K)$ by Lemma 5.2 applied to the realised rule $X \rightarrow YZ$, while the target nonterminals $[\omega(Y) : u_Y, v_Y]$ and $[\omega(Z) : u_Z, v_Z]$ belong to $\hat{V}(K)$ by the preceding application of Lemma 4.8. Therefore Lemma 5.3, which is just Rule (2), gives

$$[\omega(Y) : u_X, \omega(Z)v_X] \Rightarrow_{\hat{G}(K)} [\omega(Y) : u_Y, v_Y]$$

and

$$[\omega(Z) : u_X\omega(Y), v_X] \Rightarrow_{\hat{G}(K)} [\omega(Z) : u_Z, v_Z].$$

By the induction hypothesis, $[\omega(Y) : u_Y, v_Y] \Rightarrow_{\hat{G}(K)}^* w_1$ and $[\omega(Z) : u_Z, v_Z] \Rightarrow_{\hat{G}(K)}^* w_2$. Combining these derivations, we obtain $[\omega(X) : u_X, v_X] \Rightarrow_{\hat{G}(K)}^* w_1w_2 = w$, proving the claim.

Now let $w \in L$. If $w = \lambda$, then $\lambda \in \text{CS}(\tilde{G}) \subseteq K$, so by Rule (5) we have $\hat{S} \rightarrow \lambda$. Suppose next that $w \neq \lambda$. Choose $S_0 \rightarrow A \in P$ such that $A \Rightarrow_{\tilde{G}}^* w$, and put $p := h(w)$. As in the proof of Lemma 4.5(iii), the full typed refinement has the rule $\tilde{S} \rightarrow A_p^{e_M, e_M}$ and a derivation $A_p^{e_M, e_M} \Rightarrow_{\tilde{G}_{\text{full}}}^* w$; hence the typed symbol $X := A_p^{e_M, e_M}$ is reachable and productive and survives in the trimmed grammar $\tilde{G}$, with $X \Rightarrow_{\tilde{G}}^* w$. Since $\tilde{S} \rightarrow X$ is a rule of $\tilde{G}$, we have $\chi(X) = \langle \lambda, \lambda \rangle$. By Lemma 4.8, $\omega(X) \in \text{CS}(\tilde{G}) \subseteq K$, and therefore Rule (5) gives $\hat{S} \rightarrow [\omega(X) : \lambda, \lambda]$. Applying the claim, we obtain $[\omega(X) : \lambda, \lambda] \Rightarrow_{\hat{G}(K)}^* w$, and hence $w \in L(\hat{G}(K))$. $\square$

Theorem 5.6. *If $K \subseteq L$ is finite, then $L(\hat{G}(K)) \subseteq L$.*

Proof. We prove the following claim by induction on the derivation height in $\hat{G}(K)$.

Claim. If $[x : u, v] \Rightarrow_{\hat{G}(K)}^* w$, then $uwv \in L$ and $h(w) = h(x)$.

If the first rule is Rule (4), then the whole derivation consists of that single step, so $x = a = w$. Therefore $uwv = uav \in K \subseteq L$ and $h(w) = h(x)$.

If the first rule is Rule (2), then $[x : u, v] \rightarrow [x : u', v'] \Rightarrow_{\hat{G}(K)}^* w$. By the induction hypothesis, $u'wv' \in L$ and $h(w) = h(x)$. Also, since $u'xv', uxv \in K \subseteq L$, we have $(u', v') \in \mathcal{D}_L(x) \cap \mathcal{D}_L(w)$. By $h(x) = h(w)$ and the $\sim_h$ -substitutability of L , it follows that $\mathcal{D}_L(x) = \mathcal{D}_L(w)$. Hence $(u, v) \in \mathcal{D}_L(w)$, that is, $uwv \in L$.

If the first rule is Rule (3), then $[x : u, v] \rightarrow [x' : u, v] \Rightarrow_{\hat{G}(K)}^* w$ with $h(x) = h(x')$. By the induction hypothesis, $uwv \in L$ and $h(w) = h(x') = h(x)$.

If the first rule is Rule (1), then $[xy : u, v] \rightarrow [x : u, yv][y : ux, v] \Rightarrow_{\hat{G}(K)}^* w_1w_2 = w$ with $[x : u, yv] \Rightarrow_{\hat{G}(K)}^* w_1$ and $[y : ux, v] \Rightarrow_{\hat{G}(K)}^* w_2$. By the induction hypothesis, $uw_1yv \in L$, $uxw_2v \in L$, $h(w_1) = h(x)$, and $h(w_2) = h(y)$, so $h(w) = h(w_1)h(w_2) = h(x)h(y) = h(xy)$. Also, since $uxyv \in K \subseteq L$, we have $(u, yv) \in \mathcal{D}_L(x) \cap \mathcal{D}_L(w_1)$. By $h(x) = h(w_1)$ and $\sim_h$ -substitutability, it follows that $\mathcal{D}_L(x) = \mathcal{D}_L(w_1)$. Hence $(u, w_2v) \in \mathcal{D}_L(x) = \mathcal{D}_L(w_1)$, and therefore $uw_1w_2v \in L$. This proves the claim.

⁴Here $X \rightarrow YZ$ is a typed rule, so there exist $m, n, p, q, r \in M$ such that $X = A_p^{m, n}$, $Y = B_q^{m, rn}$, and $Z = C_r^{mq, n}$, with $qr = p$. On the other hand, by Lemma 4.6, we have $h(\omega(X)) = p$, $h(\omega(Y)) = q$, and $h(\omega(Z)) = r$. Therefore $h(\omega(X)) = p = qr = h(\omega(Y))h(\omega(Z)) = h(\omega(Y)\omega(Z))$.

Now let $w \in L(\hat{G}(K))$. If $w = \lambda$, then $\lambda \in K \subseteq L$. Otherwise, by Rule (5), there exists $x \in K$ such that $\hat{S} \rightarrow [x : \lambda, \lambda] \Rightarrow_{\hat{G}(K)}^* w$. Applying the claim with $u = v = \lambda$, we obtain $w \in L$. $\square$

Theorem 5.7 (Exact reconstruction from finite typed data). *If K is finite and satisfies $\text{CS}(\tilde{G}) \subseteq K \subseteq L$, then $L(\hat{G}(K)) = L$. Equivalently, whenever a finite sample exposes the finite typed reconstruction basis of $\tilde{G}$, the canonical learner grammar $\hat{G}(K)$ exactly reconstructs the target language.*

Proof. This follows by combining Theorem 5.5 and Theorem 5.6. $\square$

5.4 Identification in the limit

Corollary 5.8. *Every language in $\mathcal{C}_h^{\text{cf}}$ is identifiable in the limit from positive data by $\mathcal{A}_h$.*

Proof. Let $L \in \mathcal{C}_h^{\text{cf}}$, and let $\tilde{G}$ be the typed grammar of a reduced SSBNF grammar for L . By Theorem 5.7, we have $L(\hat{G}(K)) = L$ for every finite sample K satisfying $\text{CS}(\tilde{G}) \subseteq K \subseteq L$. Every text for L eventually contains every word in the finite set $\text{CS}(\tilde{G})$, and therefore the learner $\mathcal{A}_h$ eventually stabilizes to a correct hypothesis for L . $\square$

6 Computational Consequences of Fixed- h Reconstruction

In this section we establish the computational part of Theorem 3.7. The main result is that $\hat{G}(K)$ can be constructed and updated in time polynomial in $\|K\|$. As noted in Remark 3.9, this does not imply a polynomial upper bound on the size of characteristic samples for the general class $\mathcal{C}_h^{\text{cf}}$; that stronger result is obtained only for the linear subclass in Section 7.

6.1 Polynomial-time construction of the canonical hypothesis

Recall that for a finite set $K \subseteq \Sigma^*$, the learner outputs the grammar $\hat{G}(K) = (\hat{V}(K) \cup \{\hat{S}\}, \Sigma, \hat{P}(K), \hat{S})$ determined by $\hat{V}(K) := \{[x : u, v] \mid uxv \in K\}$. As above, we measure the size of a finite sample by $\|K\| := \sum_{w \in K} |w|$, and write $\ell_{\max} := \max\{|w| \mid w \in K\}$.

Theorem 6.1. *For fixed h , the grammar $\hat{G}(K)$ can be constructed from a finite sample K in time $O(\|K\|^5)$.*

Proof. Since h is fixed, the finite monoid M , its multiplication table, and the values $h(a)$ for $a \in \Sigma$ are given as part of the background data. Hence for any $w \in \Sigma^*$, the value $h(w)$ can be computed in time linear in $|w|$.

To make the construction precise, we first specify how elements of $\hat{V}(K)$ are represented. For each component string u , x , and v appearing in a factorization $uxv = w$ with $w \in K$, we maintain a dictionary **Comp** keyed by the string itself, and assign a unique integer identifier $\text{id}(z)$ to each distinct registered string z . If **Comp** is implemented as a trie over the finite alphabet Σ , then lookup and insertion for a string z take time $O(|z|)$. Thereafter, each nonterminal $[x : u, v]$ is stored as the integer triple $(\text{id}(x), \text{id}(u), \text{id}(v))$. Consequently, once identifiers have been assigned, equality of two components can be tested in constant time by comparing their identifiers.

We first construct $\hat{V}(K)$. For each $w \in K$, we enumerate all factorizations $uxv = w$, and for each such factorization append the candidate triple $(\text{id}(x), \text{id}(u), \text{id}(v))$ to a list $\mathcal{L}$. A word of length m has at most $(m+1)(m+2)/2 = O(m^2)$ factorizations, so the total number of candidates satisfies $|\mathcal{L}| = O(\sum_{w \in K} |w|^2) = O(\|K\|^2)$. For each factorization, we perform three trie lookups or insertions, and the sum of the component lengths is $|u| + |x| + |v| = |w|$. Therefore the total

cost over all factorizations arising from a word of length m is $O(m^3)$. It follows that the total time needed to generate $\mathcal{L}$ and build the component dictionary **Comp** is $O(\sum_{w \in K} |w|^3) \leq O(\|K\|^3)$.

Next, we sort $\mathcal{L}$ in lexicographic order on integer triples and remove duplicates to obtain $\hat{V}(K)$. Since each coordinate is an integer of magnitude at most $|\mathcal{L}|$, this sorting⁵ can be carried out in time $O(|\mathcal{L}|)$. Thus the entire construction of $\hat{V}(K)$ is completed in time $O(\|K\|^3)$. In particular, $|\hat{V}(K)| = O(|\mathcal{L}|) = O(\|K\|^2)$.

We now estimate the five rule families in turn.

Rule family (1). A rule $[xy : u, v] \rightarrow [x : u, yv] [y : ux, v]$ is uniquely determined by an ordered pair of children $[x : u, s], [y : t, v] \in \hat{V}(K)$ satisfying $s = yv$ and $t = ux$. Since the component strings themselves are stored in **Comp**, the equalities $s = yv$ and $t = ux$ can be checked naively by reading the stored strings, each in time $O(\ell_{\max})$. Hence the test for a single ordered pair costs $O(\ell_{\max})$. Moreover, if these equalities hold, then $[x : u, s] \in \hat{V}(K)$ implies $uxs \in K$, and since $s = yv$, it follows that $uxyv \in K$; therefore the parent $[xy : u, v]$ automatically belongs to $\hat{V}(K)$. Thus no separate membership test for the parent is needed. It follows that rule family (1) can be generated in time $O(|\hat{V}(K)|^2 \ell_{\max})$ by scanning all ordered pairs in $\hat{V}(K)$.

Rule family (2). A rule $[x : u, v] \rightarrow [x : u', v']$ corresponds exactly to an ordered pair of nonterminals having the same first component x . We therefore radix-sort $\hat{V}(K)$ by the first coordinate $\text{id}(x)$ and group together consecutive elements with the same first coordinate. The sort itself takes time $O(|\hat{V}(K)|)$. If the block sizes are $m_1, \dots, m_r$, then the number of rules generated from the i th block is m_i^2 . Hence the total time to generate rule family (2) is $O(|\hat{V}(K)| + \sum_{i=1}^r m_i^2) \leq O(|\hat{V}(K)|^2)$. Here equality of first components is tested in constant time by comparing identifiers.

Rule family (3). First, for each distinct string x that actually occurs as a first component of some element of $\hat{V}(K)$, we compute $h(x)$ once and cache the value under the corresponding identifier. There are at most $|\hat{V}(K)|$ such strings, and each satisfies $|x| \leq \ell_{\max}$, so this preprocessing takes time $O(|\hat{V}(K)| \ell_{\max})$.

Next, we radix-sort $\hat{V}(K)$ by the second and third coordinates, that is, by $(\text{id}(u), \text{id}(v))$, and group together the elements having the same outer context pair (u, v) . This sort takes time $O(|\hat{V}(K)|)$. Within each block, we further partition by the cached value $h(x)$ of the first component x . Since the monoid M is a fixed finite set, this further partitioning can be done in time linear in the size of each block. If, within all (u, v) -blocks, the sizes of the subblocks with equal $h(x)$ are $n_1, \dots, n_s$, then the number of rules generated from the j th subblock is n_j^2 . Hence the total time to generate rule family (3) is $O(|\hat{V}(K)| \ell_{\max} + |\hat{V}(K)| + \sum_{j=1}^s n_j^2) \leq O(|\hat{V}(K)| \ell_{\max} + |\hat{V}(K)|^2)$.

Rule family (4). For each $[a : u, v] \in \hat{V}(K)$, we insert the terminal rule $[a : u, v] \rightarrow a$. It suffices to arrange that the stored string data in **Comp** allow us to read in constant time whether the first component has length 1, and if so, which $a \in \Sigma$ it is. Thus rule family (4) can be generated in time $O(|\hat{V}(K)|)$.

Rule family (5). For each $w \in K$, we insert the start rule $\hat{S} \rightarrow [w : \lambda, \lambda]$, and if $\lambda \in K$, we also insert $\hat{S} \rightarrow \lambda$. Therefore rule family (5) can be generated in time $O(|K|)$.

Combining these bounds, the total construction time for $\hat{G}(K)$ is $O(\|K\|^3 + |\hat{V}(K)|^2 \ell_{\max} + |\hat{V}(K)| \ell_{\max} + |\hat{V}(K)|^2 + |K|)$. Since $|\hat{V}(K)| = O(\|K\|^2)$ and $\ell_{\max} \leq \|K\|$, the right-hand side is absorbed into $O(\|K\|^5)$. This proves the theorem. $\square$

⁵Each candidate is represented by an integer triple (i, j, k) . A standard comparison-based sort would require $O(|\mathcal{L}| \log |\mathcal{L}|)$ time, but since the range of each coordinate consists of at most $|\mathcal{L}|$ identifiers, the triples can instead be sorted in linear time by radix sort using counting sort. More precisely, one applies a stable counting sort three times, first on the third coordinate, then on the second, and finally on the first. Because stability preserves the relative order of elements with equal key values, the order established at earlier stages is preserved at later stages. Hence the final result is the lexicographic order on the full triples (i, j, k) . Since each counting sort runs in $O(|\mathcal{L}|)$ time, the total sorting time is also $O(|\mathcal{L}|)$.

Corollary 6.2. *When a new sample word z is added, the updated hypothesis $\hat{G}(K \cup \{z\})$ can be computed in time $O((\|K\| + |z|)^5)$.*

Proof. Apply Theorem 6.1 to the finite sample $K \cup \{z\}$. $\square$

6.2 The computational part of the fixed- h theorem

Corollary 6.3. *For fixed h , the learner $\mathcal{A}_h$ constructs and updates the current hypothesis in time polynomial in the size of the observed sample.*

Proof. By definition, on a finite text prefix $(w_1, \dots, w_n)$ the learner outputs $\mathcal{A}_h(w_1, \dots, w_n) = \hat{G}(\{w_1, \dots, w_n\})$. The construction bound is given by Theorem 6.1, and the update bound by Corollary 6.2. $\square$

Combining Corollary 6.3 with Corollary 5.8 yields Theorem 3.7. Thus the positive result for the general fixed- h context-free setting has a two-layer structure: exact recoverability from any finite sample exposing $\text{CS}(\tilde{G})$, and polynomial-time construction and update of the corresponding hypothesis grammar.

Remark 6.4. *The upper bound in Theorem 6.1 is a safe polynomial upper bound and is not intended to be optimal. In particular, the enumeration of rule family (1) could likely be organized more efficiently than in the coarse worst-case analysis used here. Since the purpose of this section is to establish polynomial-time constructibility of fixed- h hypotheses, we do not pursue sharper bounds.*

7 The Linear Subclass: A Full Polynomial Time-and-Data Theorem

For the linear subclass $\mathcal{C}_h^{\text{lin}}$, the obstacle identified in Remark 3.9 disappears. The constrained spine structure of linear derivations permits polynomial control over both the size of the characteristic sample and the lengths of its words. This section proves a full polynomial time-and-data theorem for $\mathcal{C}_h^{\text{lin}}$. The organisation is as follows. Section 7.1 introduces the standard normal form for linear grammars (SSLNF). Section 7.2 introduces the typed linear refinement H and establishes length bounds on the characteristic sample. Section 7.3 proves the exact reconstruction lemma for the linear case. Section 7.4 derives the polynomial time-and-data theorem.

7.1 Start-Symbol-Separated Strict Linear Normal Form

We use the following standard normal form for linear grammars.

Definition 7.1 (SSLNF). *A context-free grammar $G = (V, \Sigma, P, S_0)$ is in start-symbol-separated strict linear normal form (SSLNF) if every production has one of the forms $A \rightarrow aB$, $A \rightarrow Ba$, $A \rightarrow a$, $S_0 \rightarrow A$, or $S_0 \rightarrow \lambda$ (with $A, B \in V \setminus \{S_0\}$ and $a \in \Sigma$), and S_0 does not appear in the right-hand side of any production.*

Proposition 7.2. *Every nonempty linear⁶ context-free language has an equivalent reduced⁷ SSLNF grammar, and such a grammar can be computed in polynomial time.*

⁶A context-free grammar $G = (V, \Sigma, P, S)$ is *linear* if every rule in P is of the form $A \rightarrow uBv$ or $A \rightarrow w$, where $A, B \in V$ and $u, v, w \in \Sigma^*$.

⁷A context-free grammar is *reduced* if every nonterminal is reachable from the start symbol and productive; see Definition 4.2.

Proof. Let L be a nonempty linear context-free language. Start with any linear grammar generating L , and remove all unreachable and unproductive symbols to obtain a reduced linear grammar $G_1 = (V_1, \Sigma, P_1, S_1)$ generating the same language.

Next, introduce a fresh start symbol $S_0 \notin V_1$ and add the rule $S_0 \rightarrow S_1$. If necessary, we will later also allow $S_0 \rightarrow \lambda$, but we add no other start rules. At this point, the start symbol S_0 does not appear on any right-hand side.

After that, eliminate ε -rules for non-start symbols by the standard construction, and then eliminate unit rules $A \rightarrow B$ between non-start symbols, again by the standard construction. The crucial point here is that originally each right-hand side contains at most one nonterminal, so after performing these standard transformations, every non-start rule still has at most one nonterminal on its right-hand side. Hence linearity is preserved. Moreover, since the eliminations are restricted to non-start symbols, start-symbol separation is not destroyed. As a result, in polynomial time we obtain a grammar $G_2 = (V_2, \Sigma, P_2, S_0)$ generating the same language L , such that every non-start rule is of one of the following forms:

- (1) $A \rightarrow uBv$, where $B \in V_2 \setminus \{S_0\}$ and $|u| + |v| \geq 1$;
- (2) $A \rightarrow w$, where $w \in \Sigma^+$.

We now refine each non-start rule of G_2 into SSLNF form.

Step 1: refining a rule $A \rightarrow uBv$. Write $u = a_1 \cdots a_k$ and $v = b_1 \cdots b_\ell$. By assumption, $k + \ell \geq 1$. We replace this rule by a sequence of SSLNF rules, according to cases.

Case $k = 0$. Then $\ell \geq 1$, and the right-hand side is $Bv = Bb_1 \cdots b_\ell$.

If $\ell = 1$, then the rule is already of the form $A \rightarrow Bb_1$, which is an allowed SSLNF rule of type $X \rightarrow Ya$, so nothing needs to be done.

Now suppose $\ell \geq 2$. Introduce fresh nonterminals $D_1, \dots, D_{\ell-1}$, and replace the original rule $A \rightarrow Bb_1 \cdots b_\ell$ by the chain

$$A \rightarrow D_{\ell-1}b_\ell, \quad D_{\ell-1} \rightarrow D_{\ell-2}b_{\ell-1}, \quad \dots, \quad D_2 \rightarrow D_1b_2, \quad D_1 \rightarrow Bb_1.$$

All of these are of type $X \rightarrow Ya$. Moreover, for each $i = 1, \dots, \ell - 1$, one proves by induction on i that $D_i \Rightarrow^* Bb_1 \cdots b_i$. Indeed, for $i = 1$ this is exactly the rule $D_1 \rightarrow Bb_1$. If $i \geq 2$, then from $D_i \rightarrow D_{i-1}b_i$ and the induction hypothesis we obtain $D_i \Rightarrow^* Bb_1 \cdots b_{i-1}b_i = Bb_1 \cdots b_i$. Therefore $A \Rightarrow D_{\ell-1}b_\ell \Rightarrow^* Bb_1 \cdots b_{\ell-1}b_\ell = Bv$, so this replacement has exactly the same local effect as the original rule.

Case $k \geq 1$. Then the right-hand side is $uBv = a_1 \cdots a_k Bb_1 \cdots b_\ell$. We first build a local chain representing the part Bv , that is, the nonterminal B followed by the right suffix v .

If $\ell = 0$, define simply $D_0 := B$. This is not the introduction of a new symbol; it is merely a notational convention reflecting the fact that in this case $Bv = B$.

If $\ell \geq 1$, introduce fresh nonterminals $D_1, \dots, D_\ell$ and define

$$D_1 \rightarrow Bb_1, \quad D_2 \rightarrow D_1b_2, \quad \dots, \quad D_\ell \rightarrow D_{\ell-1}b_\ell.$$

Then for each $i = 1, \dots, \ell$, one proves by induction on i that $D_i \Rightarrow^* Bb_1 \cdots b_i$. Hence $D_\ell \Rightarrow^* Bv$. In the case $\ell = 0$, we have by definition $D_0 = B = Bv$. Thus in every case, the symbol D_ℓ (with the understanding that $D_0 = B$ when $\ell = 0$) represents exactly Bv .

We now prepend the left prefix $u = a_1 \cdots a_k$ one symbol at a time.

If $k = 1$, replace the original rule by the single rule $A \rightarrow a_1 D_\ell$. This is an allowed SSLNF rule of type $X \rightarrow aY$. Since $D_\ell \Rightarrow^* Bv$, we obtain $A \Rightarrow a_1 D_\ell \Rightarrow^* a_1 Bv = uBv$.

Now suppose $k \geq 2$. Introduce fresh nonterminals $C_1, \dots, C_{k-1}$, and replace the rule by

$$A \rightarrow a_1 C_1, \quad C_1 \rightarrow a_2 C_2, \quad \dots, \quad C_{k-2} \rightarrow a_{k-1} C_{k-1}, \quad C_{k-1} \rightarrow a_k D_\ell.$$

All of these are of type $X \rightarrow aY$. Moreover, for each $i = 1, \dots, k - 1$, one proves by backward induction that $C_i \Rightarrow^* a_{i+1} \cdots a_k D_\ell$. Indeed, $C_{k-1} \rightarrow a_k D_\ell$ is the defining rule, and for $i \leq k - 2$

one uses $C_i \rightarrow a_{i+1}C_{i+1}$ together with the induction hypothesis. Therefore $A \Rightarrow a_1C_1 \Rightarrow^* a_1 \cdots a_k D_\ell \Rightarrow^* a_1 \cdots a_k Bv = uBv$. So in this case as well, the replacement has exactly the same local effect as the original rule.

We have thus shown that every rule $A \rightarrow uBv$ can be replaced by a chain of SSLNF rules whose local derivational effect is exactly the original right-hand side uBv .

Step 2: refining a rule $A \rightarrow w$. Now consider a rule $A \rightarrow w$, and write $w = a_1 \cdots a_m \in \Sigma^+$.

If $m = 1$, then the rule is already of the form $A \rightarrow a_1$, which is an allowed SSLNF terminal rule, so nothing needs to be done.

If $m \geq 2$, introduce fresh nonterminals $E_1, \dots, E_{m-1}$ and replace the original rule by

$$A \rightarrow a_1 E_1, \quad E_1 \rightarrow a_2 E_2, \quad \dots, \quad E_{m-2} \rightarrow a_{m-1} E_{m-1}, \quad E_{m-1} \rightarrow a_m.$$

Here the initial rules are all of type $X \rightarrow aY$, and the final rule is of type $X \rightarrow a$. Also, for each $i = 1, \dots, m-1$, one proves by backward induction that $E_i \Rightarrow^* a_{i+1} \cdots a_m$. Hence $A \Rightarrow a_1 E_1 \Rightarrow^* a_1 \cdots a_m = w$, so this replacement again has exactly the same local effect as the original rule.

Step 3: verification that the language is preserved. In all of the above replacements, the auxiliary nonterminals introduced for a given original rule ρ are used only in the local chain replacing that rule. In other words, an auxiliary nonterminal introduced for one rule never appears in the replacement of any other rule. Therefore, each use of an original rule corresponds exactly to a finite local derivation along the associated replacement chain in the transformed grammar. Conversely, any derivation segment involving auxiliary nonterminals in the transformed grammar must remain inside the replacement chain to which those auxiliary symbols belong, and thus contracts uniquely to a single use of the corresponding original rule.

Applying this correspondence node by node to derivation trees, any derivation tree of the original grammar G_2 can be expanded into a derivation tree of the transformed grammar, and conversely any derivation tree of the transformed grammar can be folded back into a derivation tree of G_2 by collapsing each local chain into the original rule it replaces. Hence the transformed grammar generates the same language as G_2 .

Step 4: verification that the resulting grammar is in SSLNF. In the transformed grammar, every non-start rule is of one of the forms $X \rightarrow aY$, $X \rightarrow Ya$, or $X \rightarrow a$. On the other hand, the start rules are only of the form $S_0 \rightarrow A$, together with $S_0 \rightarrow \lambda$ if needed, as arranged in the initial preprocessing stage. Also, the start symbol S_0 never appears on a right-hand side. Therefore the resulting grammar is in SSLNF.

Step 5: reduction. Finally, remove unreachable and unproductive symbols once again. This operation only deletes unnecessary symbols and rules and does not change the form of the remaining rules, so the SSLNF property is preserved. The resulting grammar is therefore a reduced SSLNF grammar generating L .

Complexity. The initial reduction, start-symbol separation, elimination of non-start ε -rules, and elimination of non-start unit rules are all standard polynomial-time procedures. In the final refinement stage, for each rule $A \rightarrow uBv$, the number of auxiliary symbols and rules added is linear in $|u| + |v|$, and for each rule $A \rightarrow w$, the number added is linear in $|w|$. Hence the total number of added symbols and rules is linear in the total right-hand-side length of the rules of P_2 . Therefore the refinement stage as a whole is polynomial in the size of the grammar. Combining all steps, the required reduced SSLNF grammar can be computed in polynomial time.

This proves the proposition. $\square$

7.2 Typed Reconstruction Data for the Linear Case

Fix a reduced SSLNF grammar $G_0 = (V_0, \Sigma, P_0, S_0)$ for some $L \in \mathcal{C}_h^{\text{lin}}$. In this section we work with a typed refinement H that specialises the general construction of Section 4 to the

linear setting. Concretely, H is obtained by applying the typed-refinement construction ($\tilde{G}$ in Section 4) directly to G_0 ; since G_0 is in SSLNF, the non-start productions take restricted forms and only the following typed rule instances appear:

- If $A \rightarrow aB \in P_0$ with $h(a) = s$ and $s \cdot q = p$, then for all $m, n, q \in M$: $A_p^{m,n} \rightarrow a B_q^{ms,n}$;
- If $A \rightarrow Ba \in P_0$ with $h(a) = s$ and $q \cdot s = p$, then for all $m, n, q \in M$: $A_p^{m,n} \rightarrow B_q^{m,sn} a$ (here $B_q^{m,sn}$ carries the linear spine and a is the trailing terminal);
- If $A \rightarrow a \in P_0$ with $h(a) = p$, then for all $m, n \in M$: $A_p^{m,n} \rightarrow a$;
- If $S_0 \rightarrow A \in P_0$, then for all $p \in M$: $S_H \rightarrow A_p^{e_M, e_M}$;
- If $S_0 \rightarrow \lambda \in P_0$, then $S_H \rightarrow \lambda$.

Finally, remove all unreachable and unproductive symbols and rules to reduce H . This removal preserves linearity and the SSLNF rule forms. (The rule shape $A \rightarrow Ba$ is the mirror image of $A \rightarrow aB$; the arguments below treat both cases symmetrically.)

To avoid notational conflict with $\tilde{G}$ from Section 4, we write H for the typed linear grammar throughout this section. Let W denote the set of reachable and productive typed nonterminals of H , and let R denote the set of realised non-start productions of H .

All non-start productions of H have the form $X \rightarrow aY$, $X \rightarrow Ya$, or $X \rightarrow a$. Hence every derivation has a unique *spine*, i.e., a unique chain of nonterminals carrying the recursive unfolding.

For each $X \in W$ let $\omega(X)$ denote the shortlex-minimal terminal string derivable from X , and let $\chi(X) = \langle u_X, v_X \rangle$ denote the shortlex-minimal context pair such that $S_H \Rightarrow_H^* u_X X v_X$. These are defined exactly as ω and χ in Section 4 and are well-defined and finite under the linear structure of H (cf. Theorem 4.9).

The following lemma isolates the only non-trivial point needed for the linear length bounds.

Lemma 7.3 (Cycle deletion along a strict linear spine). *Let $Z_0 \Rightarrow Z_1 \Rightarrow \dots \Rightarrow Z_t$ be the spine of a strict linear derivation in H . If $Z_i = Z_j$ for some $i < j$, then the segment from Z_i to Z_j can be deleted to obtain another strict linear derivation whose spine is shorter by $j - i$ steps. However, since the terminal string carried by the deleted segment is omitted, the yield of the resulting derivation is a proper subword of the original yield and is in general different.*

Remark 7.4. *This lemma does not claim yield-equivalence. In Lemmas 7.5 and 7.6 we apply cycle deletion to a derivation chosen to minimise the yield length or context length; the deletion then produces a shorter derivation, contradicting minimality. The role of this lemma is to make explicit how such a contradiction is constructed.*

Proof. Suppose $Z_i = Z_j$ with $i < j$. Since H is in strict linear normal form, each spine step $Z_k \Rightarrow Z_{k+1}$ is induced by a rule of the form $Z_k \rightarrow aZ_{k+1}$ or $Z_k \rightarrow Z_{k+1}a$ ($a \in \Sigma$), so each step introduces exactly one terminal into the external context. Thus the segment $Z_i \Rightarrow \dots \Rightarrow Z_j$ carries a nonempty terminal string $s \in \Sigma^+$ with $|s| = j - i$.

Since $Z_i = Z_j$, the suffix derivation starting at Z_j can equally well start at Z_i . Hence

$$Z_0 \Rightarrow \dots \Rightarrow Z_i \Rightarrow Z_{j+1} \Rightarrow \dots \Rightarrow Z_t$$

is a valid strict linear derivation in H whose spine is shorter by exactly $j - i$ steps. The terminal string s carried by the deleted segment $[i, j)$ does not appear in the new derivation, so the yield of the new derivation is obtained from the original yield by removing s . $\square$

Lemma 7.5. *For every $X \in W$ there exists a derivation from X to $\omega(X)$ whose spine visits no typed nonterminal twice. Consequently $|\omega(X)| \leq |W|$.*

Proof. Choose a terminal derivation $X \Rightarrow_H^* w$ minimising $|w|$; by definition $w = \omega(X)$. Since H is strictly linear, the derivation has a unique spine $X = X_0 \Rightarrow X_1 \Rightarrow \dots \Rightarrow X_{t-1} \Rightarrow a_t$, with each non-base step contributing exactly one terminal symbol, so $|w| = t$. If some typed nonterminal is repeated, say $X_i = X_j$ with $0 \leq i < j \leq t - 1$, then by Lemma 7.3 we can delete

the segment $[i, j]$ and obtain a shorter terminal derivation from X_i , hence from X , contradicting the minimality of $|w|$. Therefore the spine is simple and $t \leq |W|$, giving $|\omega(X)| = t \leq |W|$. $\square$

Lemma 7.6. *For every $X \in W$ there exists a derivation witnessing $\chi(X) = \langle u_X, v_X \rangle$ whose spine visits no typed nonterminal twice before reaching X . Consequently $|u_X| + |v_X| \leq |W| - 1$.*

Proof. Choose a derivation $S_H \Rightarrow_H^* uXv$ minimising $|u| + |v|$; by definition $\langle u, v \rangle = \chi(X)$. Consider the typed spine $Y_0 \Rightarrow Y_1 \Rightarrow \dots \Rightarrow Y_m = X$ below the start production. Since H is strictly linear, each step $Y_i \Rightarrow Y_{i+1}$ adds exactly one terminal to the external context (on the left or right), so $|u| + |v| = m$. If $Y_i = Y_j$ for some $0 \leq i < j \leq m$, then by Lemma 7.3 we can delete the segment from Y_i to Y_j , obtaining another derivation of the same form with strictly smaller $|u| + |v|$, contradicting minimality. Hence the spine is simple. The simple spine $Y_0, Y_1, \dots, Y_m$ consists of pairwise distinct elements of W , so $m + 1 \leq |W|$, i.e., $m \leq |W| - 1$. Therefore $|u_X| + |v_X| = m \leq |W| - 1$. $\square$

We now define a characteristic sample that is more compact than in the general context-free case. For each $X \in W$ set $\alpha_X := u_X \omega(X) v_X$. For each realised rule $r \in R$ choose one *rule-witness word* ρ_r as follows: if r is $X \rightarrow a$ set $\rho_r := u_X a v_X$; if r is $X \rightarrow aY$ set $\rho_r := u_X a \omega(Y) v_X$; if r is $X \rightarrow Ya$ set $\rho_r := u_X \omega(Y) a v_X$. Include λ if the start production $S_H \rightarrow \lambda$ exists, and define

$$\text{CS}_{\text{lin}}(H) := \{\alpha_X : X \in W\} \cup \{\rho_r : r \in R\} \cup (\{\lambda\} \text{ if } S_H \rightarrow \lambda \in P_H).$$

Proposition 7.7. *Let H be the typed linear grammar obtained from a reduced SSLNF grammar under fixed h . Then: (i) $|W| = O(|V_0| |M|^3)$; (ii) $|R| = O(|P_0| |M|^3)$; (iii) $|\text{CS}_{\text{lin}}(H)| \leq |W| + |R| + 1$; (iv) every word in $\text{CS}_{\text{lin}}(H)$ has length at most $2|W|$. In particular, $|\text{CS}_{\text{lin}}(H)|$ and $\max\{|w| : w \in \text{CS}_{\text{lin}}(H)\}$ are both polynomial in the size of the original linear grammar, for fixed h .*

Proof. (i): Each typed symbol has the form $A_p^{m,n}$ with $A \in V_0 \setminus \{S_0\}$ and $m, n, p \in M$, so the number of possible typed symbols is at most $|V_0| |M|^3$; reduction can only decrease this count. The rules remaining after reduction all have the form $X \rightarrow aY$, $X \rightarrow Ya$, or $X \rightarrow a$, and S_H does not appear on any right-hand side, so H remains in SSLNF and is reduced.

(ii): Each non-start SSLNF production $A \rightarrow aB$, $A \rightarrow Ba$, or $A \rightarrow a$ generates $O(|M|^3)$ typed copies (over $m, n, p \in M$ with the appropriate q determined by $h(a)$), so $|R| = O(|P_0| |M|^3)$.

(iii): By definition $\text{CS}_{\text{lin}}(H)$ contains at most one anchor α_X per $X \in W$, at most one rule-witness ρ_r per $r \in R$, and at most one additional word λ , giving $|\text{CS}_{\text{lin}}(H)| \leq |W| + |R| + 1$.

(iv): By Lemmas 7.5 and 7.6, $|\omega(X)| \leq |W|$ and $|u_X| + |v_X| \leq |W| - 1$, so $|\alpha_X| = |u_X| + |\omega(X)| + |v_X| \leq (|W| - 1) + |W| = 2|W| - 1$. If r is $X \rightarrow a$ then $|\rho_r| = |u_X| + 1 + |v_X| \leq |W|$. If r is $X \rightarrow aY$ or $X \rightarrow Ya$ then $|\rho_r| = |u_X| + |v_X| + 1 + |\omega(Y)| \leq (|W| - 1) + 1 + |W| = 2|W|$. Hence every word in $\text{CS}_{\text{lin}}(H)$ has length at most $2|W|$. The final claim follows immediately from (i)–(iv). $\square$

7.3 Exact Reconstruction for the Linear Case

The sample $\text{CS}_{\text{lin}}(H)$ plays exactly the same role for the typed linear grammar H as $\text{CS}(\tilde{G})$ (Section 4) plays in the general context-free case. To make the correspondence precise: whereas in the general case each word in $\text{CS}(\tilde{G})$ records the canonical observation $u_X \omega(Y) \omega(Z) v_X$ for a binary rule $X \rightarrow YZ$, the rule-witness word ρ_r in $\text{CS}_{\text{lin}}(H)$ corresponds to a unary-style rule $X \rightarrow aY$ or $X \rightarrow Ya$ and contains only a single terminal symbol together with one child spine ($\omega(Y)$). The absence of binary branching is what enables polynomial control over word lengths, and the exact reconstruction lemma below exploits this structural simplification.

Lemma 7.8. *Let H be the typed linear grammar obtained from a reduced SSLNF grammar. If K is finite and $\text{CS}_{\text{lin}}(H) \subseteq K \subseteq L(H)$, then $L(\hat{G}(K)) = L(H)$.*

Proof. Soundness $L(\hat{G}(K)) \subseteq L(H)$.

We prove the following claim by induction on the derivation height in $\hat{G}(K)$:

Claim. If $[x : u, v] \Rightarrow_{\hat{G}(K)}^* w$, then $uwv \in L(H)$ and $h(w) = h(x)$.

Here H is the typed refinement of L , and by Lemma 4.5(iii) we have $L(H) = L$. We also use that $K \subseteq L(H)$.

If the first rule is Rule (4), then $x = a = w$. Since $uav \in K \subseteq L(H)$, we obtain $uwv \in L(H)$. Also, $h(w) = h(a) = h(x)$.

If the first rule is Rule (2), then $[x : u, v] \rightarrow [x : u', v'] \Rightarrow_{\hat{G}(K)}^* w$. By the induction hypothesis, $u'wv' \in L(H)$ and $h(w) = h(x)$. Also, since $u'xv', uxv \in K \subseteq L(H)$, we have $(u', v') \in \mathcal{D}_{L(H)}(x)$ and $(u, v) \in \mathcal{D}_{L(H)}(x)$, and since $u'wv' \in L(H)$, we also have $(u', v') \in \mathcal{D}_{L(H)}(w)$. Hence $(u', v') \in \mathcal{D}_{L(H)}(x) \cap \mathcal{D}_{L(H)}(w)$, and by $h(x) = h(w)$ and the $\sim_h$ -substitutability of $L(H)$, it follows that $\mathcal{D}_{L(H)}(x) = \mathcal{D}_{L(H)}(w)$. Therefore $(u, v) \in \mathcal{D}_{L(H)}(w)$, that is, $uwv \in L(H)$.

If the first rule is Rule (3), then $[x : u, v] \rightarrow [x' : u, v] \Rightarrow_{\hat{G}(K)}^* w$, with $h(x) = h(x')$. By the induction hypothesis, $uwv \in L(H)$ and $h(w) = h(x')$. Hence $h(w) = h(x)$ as well.

If the first rule is Rule (1), then

$$[xy : u, v] \rightarrow [x : u, yv] [y : ux, v] \Rightarrow_{\hat{G}(K)}^* w_1 w_2 = w.$$

By the induction hypothesis, we have $uw_1 yv \in L(H)$, $uxw_2 v \in L(H)$, $h(w_1) = h(x)$, and $h(w_2) = h(y)$. Therefore $h(w) = h(w_1)h(w_2) = h(x)h(y) = h(xy)$. Moreover, since $uxyv \in K \subseteq L(H)$, we have $(u, yv) \in \mathcal{D}_{L(H)}(x)$, while $uw_1 yv \in L(H)$ gives $(u, yv) \in \mathcal{D}_{L(H)}(w_1)$. Hence $(u, yv) \in \mathcal{D}_{L(H)}(x) \cap \mathcal{D}_{L(H)}(w_1)$, and by $h(x) = h(w_1)$ and $\sim_h$ -substitutability, we obtain $\mathcal{D}_{L(H)}(x) = \mathcal{D}_{L(H)}(w_1)$. Since $uxw_2 v \in L(H)$, we also have $(u, w_2 v) \in \mathcal{D}_{L(H)}(x)$. Therefore $(u, w_2 v) \in \mathcal{D}_{L(H)}(w_1)$, and thus $uw_1 w_2 v = uwv \in L(H)$.

This proves the claim. Now let $w \in L(\hat{G}(K))$. If $w = \lambda$, then Rule (5) gives $\lambda \in K \subseteq L(H)$. If $w \neq \lambda$, then by Rule (5) there exists $x \in K$ such that

$$\hat{S} \rightarrow [x : \lambda, \lambda] \Rightarrow_{\hat{G}(K)}^* w.$$

Applying the claim with $u = v = \lambda$, we obtain $w \in L(H)$. Hence $L(\hat{G}(K)) \subseteq L(H)$.

Completeness $L(H) \subseteq L(\hat{G}(K))$.

For each $X \in W$, write $\hat{X} := [\omega(X) : u_X, v_X]$. Since $\alpha_X = u_X \omega(X) v_X \in \text{CS}_{\text{lin}}(H) \subseteq K$, we have $\hat{X} \in \hat{V}(K)$. We show that for every $X \in W$ and every terminal word z with $X \Rightarrow_H^* z$, we have $\hat{X} \Rightarrow_{\hat{G}(K)}^* z$. We prove this by induction on the length of the fixed unique spine in the strictly linear derivation in H .

If the spine length is 0, then the realised rule at the root is $X \rightarrow a$, and hence $z = a$. From the corresponding rule-evidence word $\rho_r = u_X a v_X \in K$, we obtain $[a : u_X, v_X] \in \hat{V}(K)$. Also, since a is a terminal yield of X via the rule $X \rightarrow a$, Lemma 4.6 gives $h(\omega(X)) = h(a)$. Therefore Rule (3) yields $\hat{X} \rightarrow [a : u_X, v_X]$, and Rule (4) yields $[a : u_X, v_X] \rightarrow a$. Hence $\hat{X} \Rightarrow_{\hat{G}(K)}^* a = z$.

Now assume that the claim has been proved for all derivations whose spine length is at most m , and consider a derivation of spine length $m + 1$.

Suppose first that the first realised rule is $r : X \rightarrow aY$. Then we may write $z = aw$, where $Y \Rightarrow_H^* w$. By the construction of the typed refinement, the typed instance of r has the form $X = A_p^{m_0, n_0} \rightarrow a B_q^{m_0 s, n_0}$, where $s = h(a)$ and $sq = p$. From the rule-evidence word $\rho_r = u_X a \omega(Y) v_X \in K$, we obtain

$$[a\omega(Y) : u_X, v_X], \quad [a : u_X, \omega(Y)v_X], \quad [\omega(Y) : u_X a, v_X]$$

all in $\hat{V}(K)$. Hence Rule (1) gives

$$[a\omega(Y) : u_X, v_X] \rightarrow [a : u_X, \omega(Y)v_X] [\omega(Y) : u_X a, v_X],$$

and Rule (4) gives $[a : u_X, \omega(Y)v_X] \rightarrow a$. Moreover, since $a\omega(Y)$ is also a terminal yield of X obtained by starting with the rule r , Lemma 4.6 gives $h(\omega(X)) = h(a\omega(Y))$. Therefore Rule (3) yields $\hat{X} \rightarrow [\omega(Y) : u_X, v_X]$.

Now set $\hat{Y} := [\omega(Y) : u_Y, v_Y]$. Since $\alpha_Y = u_Y\omega(Y)v_Y \in \text{CS}_{\text{lin}}(H) \subseteq K$, we have $\hat{Y} \in \hat{V}(K)$, and since $\rho_r = u_X a \omega(Y) v_X \in K$, we also have $[\omega(Y) : u_X a, v_X] \in \hat{V}(K)$. Thus Rule (2) also gives

$$[\omega(Y) : u_X a, v_X] \Rightarrow_{\hat{G}(K)} \hat{Y}.$$

By the induction hypothesis,

$$\hat{Y} \Rightarrow_{\hat{G}(K)}^* w.$$

Combining these derivations, we obtain

$$\hat{X} \Rightarrow [a\omega(Y) : u_X, v_X] \Rightarrow [a : u_X, \omega(Y)v_X] [\omega(Y) : u_X a, v_X] \Rightarrow a \hat{Y} \Rightarrow_{\hat{G}(K)}^* a w = z.$$

Suppose next that the first realised rule is $r : X \rightarrow Ya$. Then we may write $z = wa$, where $Y \Rightarrow_H^* w$. From the rule-evidence word $\rho_r = u_X \omega(Y) a v_X \in K$, we obtain

$$[\omega(Y)a : u_X, v_X], \quad [\omega(Y) : u_X, av_X], \quad [a : u_X \omega(Y), v_X]$$

all in $\hat{V}(K)$. Hence Rule (1) gives

$$[\omega(Y)a : u_X, v_X] \rightarrow [\omega(Y) : u_X, av_X] [a : u_X \omega(Y), v_X],$$

and Rule (4) gives $[a : u_X \omega(Y), v_X] \rightarrow a$. Moreover, since $\omega(Y)a$ is also a terminal yield of X obtained by starting with the rule r , Lemma 4.6 gives $h(\omega(X)) = h(\omega(Y)a)$. Therefore Rule (3) yields $\hat{X} \rightarrow [\omega(Y)a : u_X, v_X]$.

Since $\alpha_Y = u_Y \omega(Y) v_Y \in \text{CS}_{\text{lin}}(H) \subseteq K$, we have $\hat{Y} = [\omega(Y) : u_Y, v_Y] \in \hat{V}(K)$, and since $\rho_r \in K$, we also have $[\omega(Y) : u_X, av_X] \in \hat{V}(K)$. Thus Rule (2) also gives

$$[\omega(Y) : u_X, av_X] \Rightarrow_{\hat{G}(K)} \hat{Y}.$$

By the induction hypothesis,

$$\hat{Y} \Rightarrow_{\hat{G}(K)}^* w.$$

Combining these derivations, we obtain

$$\hat{X} \Rightarrow [\omega(Y)a : u_X, v_X] \Rightarrow [\omega(Y) : u_X, av_X] [a : u_X \omega(Y), v_X] \Rightarrow \hat{Y} a \Rightarrow_{\hat{G}(K)}^* w a = z.$$

We have therefore shown that for every $X \in W$ and every terminal word z with $X \Rightarrow_H^* z$, we have $\hat{X} \Rightarrow_{\hat{G}(K)}^* z$.

Finally, let $z \in L(H)$. If $z = \lambda$, then $\lambda \in \text{CS}_{\text{lin}}(H) \subseteq K$, so Rule (5) gives $\hat{S} \rightarrow \lambda$. If $z \neq \lambda$, choose a child X of a start rule such that $\chi(X) = \langle \lambda, \lambda \rangle$. Then $\hat{X} = [\omega(X) : \lambda, \lambda]$, and since $\alpha_X = \omega(X) \in K$, Rule (5) gives $\hat{S} \rightarrow \hat{X}$. By the claim proved above, $\hat{X} \Rightarrow_{\hat{G}(K)}^* z$, and hence $z \in L(\hat{G}(K))$. Therefore $L(H) \subseteq L(\hat{G}(K))$.

Hence $L(\hat{G}(K)) = L(H)$. □

7.4 Full Polynomial Time-and-Data Theorem

We now obtain the final strengthening of the fixed- h theory.

Theorem 7.9. *For every explicit finite monoid homomorphism $h : \Sigma^* \rightarrow M$, the learner $\mathcal{A}_h$ identifies every language in $\mathcal{C}_h^{\text{lin}}$ from positive data in polynomial time and data.*

Proof. Let $L \in \mathcal{C}_h^{\text{lin}}$, choose a reduced SSLNF grammar for L , and let H be its typed linear refinement.

We first verify $\text{CS}_{\text{lin}}(H) \subseteq L(H)$. We show that each word in $\text{CS}_{\text{lin}}(H)$ belongs to $L(H)$, by type of word.

- *Anchor words* $\alpha_X = u_X \omega(X) v_X$ ($X \in W$). Since X is reachable and productive in H , $S_H \Rightarrow_H^* u_X X v_X$ and $X \Rightarrow_H^* \omega(X)$. Concatenating, $S_H \Rightarrow_H^* u_X \omega(X) v_X$, so $\alpha_X \in L(H)$.
- *Rule-witness words* ρ_r ($r \in R$, $r : X \rightarrow a$). Since X is reachable and productive and $X \Rightarrow_H^* a$ via r , we have $S_H \Rightarrow_H^* u_X a v_X$, so $\rho_r \in L(H)$.
- *Rule-witness words* ρ_r ($r : X \rightarrow aY$). Since $Y \in W$ is productive, $Y \Rightarrow_H^* \omega(Y)$; rule r gives $X \Rightarrow_H^* a\omega(Y)$; since X is reachable, $S_H \Rightarrow_H^* u_X a\omega(Y) v_X$, so $\rho_r \in L(H)$. The case $r : X \rightarrow Ya$ is symmetric.
- *The word* λ (when $S_H \rightarrow \lambda \in \tilde{P}$). By definition $\lambda \in L(H)$.

Hence $\text{CS}_{\text{lin}}(H) \subseteq L(H) = L$.

By Lemma 7.8, $L(\hat{G}(K)) = L$ for every finite sample K with $\text{CS}_{\text{lin}}(H) \subseteq K \subseteq L(H) = L$. Thus $\text{CS}_{\text{lin}}(H)$ is a characteristic sample for L with respect to $\mathcal{A}_h$.

By Proposition 7.7, $|\text{CS}_{\text{lin}}(H)|$ and the length of each word in $\text{CS}_{\text{lin}}(H)$ are both polynomial in the size $\|G_0\| := |V_0| + |P_0| + |\Sigma|$ of the original linear grammar G_0 , for fixed h (in Proposition 7.7(i)–(iv),

$|V| = |V_0|$, $|P| = |P_0|$, and $|M|$ is a fixed constant). Hence $\|\text{CS}_{\text{lin}}(H)\| := \sum_{w \in \text{CS}_{\text{lin}}(H)} |w|$ is also polynomially bounded. By Theorem 6.1, the hypothesis grammar $\hat{G}(K) = \mathcal{A}_h(K)$ can be constructed from any finite sample K in time polynomial in $\|K\| := \sum_{w \in K} |w|$. Therefore, once the learner has seen a sample containing $\text{CS}_{\text{lin}}(H)$, it computes the correct hypothesis in time polynomial in the sample size, and the sample itself is polynomially bounded. Hence $\mathcal{A}_h$ identifies every language in $\mathcal{C}_h^{\text{lin}}$ in polynomial time and data. $\square$

Remark 7.10. *The fixed- h theory has two levels. For the full context-free class $\mathcal{C}_h^{\text{cf}}$ it yields exact reconstruction from finite typed observations, identification in the limit, and polynomial-time hypothesis construction from data. For the linear subclass $\mathcal{C}_h^{\text{lin}}$ it additionally provides polynomial bounds on both the size of the characteristic sample and the lengths of its words, establishing a full polynomial time-and-data theorem.*

8 The Strict Inclusion $\text{KL} \subsetneq \text{RS}$ at the Regular Level

The main positive theory of the paper is complete. This section and the next provide structural boundary results that locate the fixed- h framework within the broader landscape of distributional learning. These results are secondary to the main theory but clarify its scope.

We next consider the capped-counter family CCL_p . This family provides the main regular-level evidence for the strict inclusion $\text{KL} \subsetneq \text{RS}$. It makes completely explicit both the recognizing finite monoid and the automata-theoretic structure, and also prepares a sharp contrast with the uncapped language CCL considered later.

Example 8.1 (CCL_2). *Let $\Sigma = \{d, u, ;\}$, and consider a pushdown automaton that pushes on d , pops on u (rejecting if the stack is empty), and clears the entire stack on $;$. A finite automaton monitors the stack depth and rejects whenever the depth exceeds 2:*

$$0 \xrightarrow{d} 1 \xrightarrow{d} 2 \xrightarrow{d} \text{sink}, \quad 2 \xrightarrow{u} 1 \xrightarrow{u} 0, \quad 0 \xrightarrow{u} \text{sink}, \quad \{0, 1, 2\} \xrightarrow{;} 0.$$

A string is accepted iff the run from state 0 never reaches bad. For example, $du, dduu, dd; du \in \text{CCL}_2$, whereas $ddduuu, u \notin \text{CCL}_2$.

Because CCL_2 controls the stack by means of a finite-memory monitor, it collapses to a regular language. A DFA accepting CCL_2 is as follows.

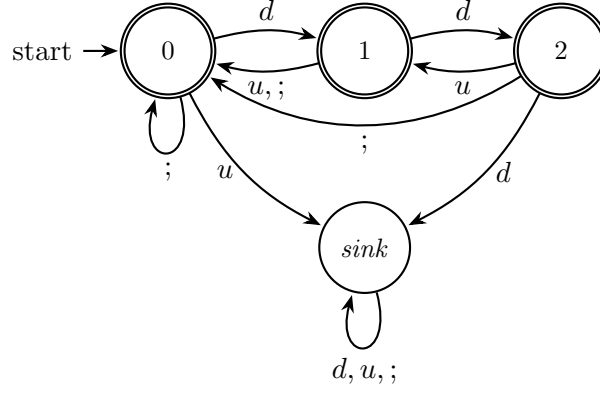


Figure 1: Finite-state depth monitor for CCL_2 , corresponding to the formal definition below with $Q_2 = \{0, 1, 2, \text{sink}\}$. Here d increases the depth, u decreases it, and $;$ resets the depth to 0. Underflow and overflow move to the absorbing state sink . Hence $\text{CCL}_2 = \{w \in \Sigma^* \mid \delta_2^*(0, w) \neq \text{sink}\}$.

This language is not (k, ℓ) -substitutable for any $k, \ell \geq 0$; see Theorem 8.6. However, the monitoring finite automaton induces exactly the algebraic information needed for $\sim$ -substitutability, namely a recognizable congruence.

Remark 8.2. *Since CCL_2 is regular, it can be written as $\text{CCL}_2 = (B_2;)^* B_2$, where $B_2 := (d(du)^*u)^*(\varepsilon \cup d(du)^*(\varepsilon \cup d))$. Equivalently, CCL_2 is generated by a right-linear grammar with nonterminals S, A, B , start symbol S , and rules $S \rightarrow \varepsilon \mid dA \mid ;S$, $A \rightarrow \varepsilon \mid uS \mid dB \mid ;S$, and $B \rightarrow \varepsilon \mid uA \mid ;S$.*

8.1 The capped-counter family

Fix the alphabet $\Sigma := \{d, u, ;\}$. For a word $w \in \Sigma^*$, define the *current depth* by reading w from left to right: d changes the depth by $+1$, u changes it by -1 , and $;$ resets the depth to 0. A run is said to be *safe up to height p* if it never lets the depth become negative and never exceeds p .

Definition 8.3 (Capped-counter language CCL_p). *For $p \geq 1$, let $\text{CCL}_p \subseteq \Sigma^*$ be the set of all words whose left-to-right run is safe up to height p . Equivalently, CCL_p is accepted by the DFA with state set $Q_p := \{0, 1, \dots, p, \text{sink}\}$, initial state 0, accepting set $\{0, 1, \dots, p\}$, and transition function δ_p*

$$\begin{aligned} \delta_p(i, d) &= \begin{cases} i+1 & (0 \leq i < p), \\ \text{sink} & (i = p \text{ or } i = \text{sink}), \end{cases} \\ \delta_p(i, u) &= \begin{cases} i-1 & (1 \leq i \leq p), \\ \text{sink} & (i = 0 \text{ or } i = \text{sink}), \end{cases} \\ \delta_p(i, ;) &= \begin{cases} 0 & (0 \leq i \leq p), \\ \text{sink} & (i = \text{sink}). \end{cases} \end{aligned}$$

Thus CCL_p consists of exactly those words whose semicolon-delimited blocks behave as counter paths staying between depth 0 and depth p .

Proposition 8.4. *For every $p \geq 1$, the language CCL_p is regular.*

Proof. This is immediate from Definition 8.3, since CCL_p is accepted by the finite automaton $(Q_p, \Sigma, \delta_p, 0, \{0, 1, \dots, p\})$. $\square$

8.2 An explicit recognizable-congruence witness

The regularity of CCL_p already places it within the recognizable-congruence framework, but it is useful to make the witness explicit. Let $A_p = (Q_p, \Sigma, \delta_p, 0, F_p)$ be the DFA of Definition 8.3, with $F_p := \{0, 1, \dots, p\}$. For a word $w \in \Sigma^*$, let $\tau_w : Q_p \rightarrow Q_p$ be the state transformation induced by w , namely $\tau_w(q) := \delta_p(q, w)$. Define the transition monoid M_p of A_p by

$$M_p := \{\tau_w \mid w \in \Sigma^*\}.$$

For transformations $\tau, \sigma \in M_p$, we write

$$\tau \cdot \sigma := \sigma \circ \tau.$$

With this convention, the identity element is $\tau_\lambda = \text{id}_{Q_p}$, and the map $h_p : \Sigma^* \rightarrow M_p$ defined by $h_p(w) := \tau_w$ is a monoid homomorphism. Indeed, for all $w, w' \in \Sigma^*$ and all $q \in Q_p$, we have

$$h_p(ww')(q) = \tau_{ww'}(q) = \delta_p(q, ww') = \delta_p(\delta_p(q, w), w') = (\tau_{w'} \circ \tau_w)(q) = (\tau_w \cdot \tau_{w'})(q),$$

and hence

$$h_p(ww') = h_p(w) \cdot h_p(w').$$

Thus a word w is finitely summarized not only by the state it reaches from the initial state, but by the full state transformation it induces on all states.

Proposition 8.5. *For every $p \geq 1$, the language CCL_p is $\sim_{h_p}$ -substitutable; in particular, $\text{CCL}_p \in \text{RS}$.*

Proof. Since the finite monoid M_p recognizes CCL_p , if we let $F'_p := \{\tau \in M_p \mid \tau(0) \in F_p\}$, then $\text{CCL}_p = h_p^{-1}(F'_p)$. Indeed, a word w is accepted iff the transformation induced by w sends 0 to a non-sink state. Hence CCL_p is recognized by $h_p : \Sigma^* \rightarrow M_p$, and Proposition 3.1 implies that CCL_p is $\sim_{h_p}$ -substitutable. $\square$

8.3 Failure of (k, ℓ) -substitutability

We now show that the bounded prefix-suffix hierarchy cannot capture the capped-counter family once the cap exceeds the trivial bound 1.

Theorem 8.6. *For every fixed $p \geq 2$, the language CCL_p is not (k, ℓ) -substitutable for any $k, \ell \geq 0$. By contrast, CCL_1 is exceptionally $(1, 1)$ -substitutable; see Appendix B.*

Proof. Fix $p \geq 2$. To show that CCL_p is not (k, ℓ) -substitutable for any $k, \ell \geq 0$, let $k, \ell \geq 0$ be arbitrary and construct witnesses contradicting Yoshinaka's definition.

Set $m := \lfloor k/2 \rfloor$ and $q := k - 2m \in \{0, 1\}$, and define $\xi := (du)^m d^q$ and $\eta := ;^\ell$. Then $|\xi| = 2m + q = k$ and $|\eta| = \ell$. In particular, if $k = 0$, then $m = q = 0$, so $\xi = \lambda$.

We first record the basic behavior of ξ . The word $(du)^m$ is a repetition of blocks that temporarily increase the depth by 1 and then immediately return it to its previous value. Hence, when read from depth 0, every prefix of $(du)^m$ stays within $\{0, 1\}$, and the final depth is 0. Appending d^q then leaves the depth unchanged if $q = 0$, and increases it by 1 if $q = 1$. Thus reading ξ from depth 0 is safe and ends at depth exactly q . Likewise, reading ξ from depth 1 never causes underflow and ends at depth $1 + q$.

Next set $a_1 := p - q - 1$ and $a_2 := p - q$, and define $y_1 := d^{a_1} u^{a_1}$ and $y_2 := d^{a_2} u^{a_2}$. Since $p \geq 2$ and $q \in \{0, 1\}$, we have $a_1 \geq 0$ and $a_2 \geq 1$. Thus y_1 and y_2 are well defined, and in particular $y_2 \neq \lambda$.

We now verify the nonemptiness conditions required in Yoshinaka's definition. First, $\xi y_2 \eta \neq \lambda$ follows immediately from $y_2 \neq \lambda$. Next consider $\xi y_1 \eta$. If $y_1 \neq \lambda$, there is nothing to prove. If

$y_1 = \lambda$, then $a_1 = 0$, so $p - q - 1 = 0$. Since $p \geq 2$, this forces $q = 1$. Hence $k = 2m + 1 \geq 1$, so $|\xi| = k \geq 1$ and therefore $\xi \neq \lambda$. Thus in this case as well, $\xi y_1 \eta = \xi \eta \neq \lambda$.

Now take $v := \xi \in \Sigma^k$ and $u := \eta \in \Sigma^\ell$ in Yoshinaka's definition; when $k = 0$, this simply means $v = \xi = \lambda \in \Sigma^0$. Also set $x_1 := \lambda$, $z_1 := \lambda$, $x_2 := d$, and $z_2 := \lambda$. We examine the four words $x_1 v y_1 u z_1$, $x_1 v y_2 u z_1$, $x_2 v y_1 u z_2$, and $x_2 v y_2 u z_2$, that is, $\xi y_1 \eta$, $\xi y_2 \eta$, $d \xi y_1 \eta$, and $d \xi y_2 \eta$. As shown above, both $\xi y_1 \eta$ and $\xi y_2 \eta$ are nonempty.

$\xi y_1 \eta \in \text{CCL}_p$. Reading ξ from depth 0 is safe and ends at depth q . Then reading $y_1 = d^{a_1} u^{a_1}$ raises the depth, during the initial block d^{a_1} , to at most $q + a_1 = q + (p - q - 1) = p - 1$, which does not exceed the bound p . The subsequent block u^{a_1} returns the depth to q . Finally, $\eta = ;^\ell$ merely resets the depth to 0 at each symbol $;$, so the computation remains safe throughout. Hence $\xi y_1 \eta \in \text{CCL}_p$.

$\xi y_2 \eta \in \text{CCL}_p$. Similarly, reading ξ from depth 0 ends at depth q . Then reading $y_2 = d^{a_2} u^{a_2}$ raises the depth, during the initial block d^{a_2} , to at most $q + a_2 = q + (p - q) = p$, which is still within the allowed range. The subsequent block u^{a_2} returns the depth to q , and finally η resets the depth to 0. Therefore $\xi y_2 \eta \in \text{CCL}_p$.

$d \xi y_1 \eta \in \text{CCL}_p$. After the initial d , the depth is 1. Reading ξ from depth 1 is safe, as noted above, and ends at depth $1 + q$. Then reading $y_1 = d^{a_1} u^{a_1}$ raises the depth, during the initial block d^{a_1} , to at most $1 + q + a_1 = 1 + q + (p - q - 1) = p$, which is again allowed. The subsequent block u^{a_1} returns the depth to $1 + q$, and finally η resets the depth to 0. Hence $d \xi y_1 \eta \in \text{CCL}_p$.

$d \xi y_2 \eta \notin \text{CCL}_p$. After reading the initial d and then ξ , the current depth is $1 + q$. Now the initial block $d^{a_2} = d^{p-q}$ of y_2 raises the depth to $1 + q + (p - q) = p + 1$. This exceeds the bound p , so the safety condition fails at that point. Therefore $d \xi y_2 \eta \notin \text{CCL}_p$.

We have thus found $v = \xi \in \Sigma^k$ and $u = \eta \in \Sigma^\ell$ such that $v y_1 u \neq \lambda$ and $v y_2 u \neq \lambda$, with $x_1 v y_1 u z_1$, $x_1 v y_2 u z_1$, and $x_2 v y_1 u z_2$ in CCL_p , but $x_2 v y_2 u z_2 \notin \text{CCL}_p$. This contradicts the assumption that CCL_p is (k, ℓ) -substitutable. Since $k, \ell \geq 0$ were arbitrary, CCL_p is not (k, ℓ) -substitutable for any $k, \ell \geq 0$. $\square$

8.4 Consequence

Corollary 8.7. *For every $p \geq 2$, $\text{CCL}_p \in \text{RS} \setminus \text{KL}$. Therefore $\text{KL} \subsetneq \text{RS}$.*

Proof. By Proposition 8.5, we have $\text{CCL}_p \in \text{RS}$ for every $p \geq 1$. By Theorem 8.6, we have $\text{CCL}_p \notin \text{KL}$ whenever $p \geq 2$. Hence $\text{CCL}_p \in \text{RS} \setminus \text{KL}$ for every $p \geq 2$, and in particular $\text{KL} \subsetneq \text{RS}$. $\square$

Remark 8.8. *The strict inclusion witnessed here is already visible at the regular level. Thus the extra expressive power of recognizable-congruence control over bounded prefix-suffix control does not first appear only in some pathological nonregular example; it is already present in a very simple finite-state counter family.*

9 Boundary Results Beyond the Capped-Counter Family

In Section 8, we showed that recognizable-congruence control is already strictly stronger than bounded prefix-suffix control at the regular level. We now turn in the opposite direction. Although RS strictly contains KL , it still does not cover all deterministic context-free languages. We show this rigorously by means of three boundary examples: the uncapped counter language CCL , the one-bracket Dyck language D_1 , and Yoshinaka's classical language $L_Y = L(S \rightarrow aSS \mid b)$.

We use the standard model of deterministic pushdown automata (DPDAs) [14]: a DPDA is a pushdown automaton in which, for every state q and stack symbol A , (i) $|\delta(q, x, A)| \leq 1$ for all $x \in \Sigma \cup \{\varepsilon\}$, and (ii) if $\delta(q, \varepsilon, A) \neq \emptyset$ then $\delta(q, a, A) = \emptyset$ for all $a \in \Sigma$. Acceptance is by final

state throughout. Throughout, we place a distinguished symbol Z_0 at the bottom of the stack; Z_0 serves as a permanent bottom-of-stack marker that is never removed by any transition.

Intuitively, we write CCL for the cap-free limit $\lim_{p \rightarrow \infty} \text{CCL}_p$. A deterministic pushdown automaton recognizes CCL as follows.

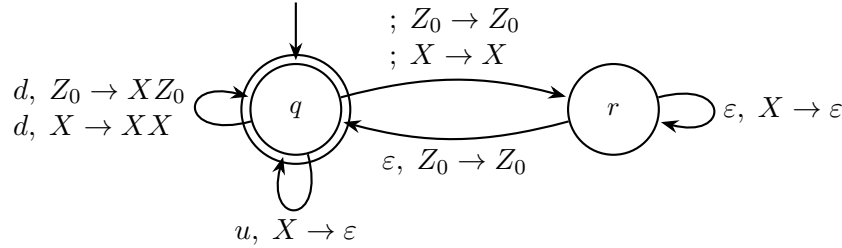


Figure 2: A DPDA for CCL. The transition $\delta(q, u, Z_0)$ is undefined, so if an underflow occurs, the computation halts and the input is rejected.

9.1 The uncapped limit CCL

We begin with the uncapped limit of the capped-counter family from Section 8. It preserves the same local mechanism as CCL_p , but removes only the finite-height restriction. We note that a *block* of a word in CCL refers to each of the subwords delimited by the separator symbol $;$.

Definition 9.1 (The uncapped-counter language CCL).

$$\text{CCL} := \{ w \in \{d, u, ;\}^* \mid \forall \text{ block } B, \forall \text{ prefix } p \text{ of } B: \#_u(p) \leq \#_d(p) \}$$

Equivalently, CCL consists of all words such that, in the left-to-right run where d increases the depth by 1, u decreases it by 1, and $;$ resets it to 0, the depth never drops below 0.

Proposition 9.2. The language CCL is deterministic context-free.

Proof. We verify that the DPDA M of Figure 2 recognises CCL. Formally, $M = (Q, \Sigma, \Gamma, \delta, q, Z_0, F)$, where

$$Q = \{q, r\}, \quad \Sigma = \{d, u, ;\}, \quad \Gamma = \{Z_0, X\}, \quad F = \{q\},$$

and the transition function δ is defined by

$$\begin{aligned} \delta(q, d, Z_0) &= (q, XZ_0), & \delta(q, d, X) &= (q, XX), \\ \delta(q, u, X) &= (q, \varepsilon), \\ \delta(q, ;, Z_0) &= (r, Z_0), & \delta(q, ;, X) &= (r, X), \\ \delta(r, \varepsilon, X) &= (r, \varepsilon), & \delta(r, \varepsilon, Z_0) &= (q, Z_0). \end{aligned}$$

All other transitions are undefined.

Informally, q is the normal scanning state; the number of X -symbols on the stack tracks the current depth within the active block. Reading d pushes one X ; reading u pops one X ; reading $;$ enters state r , which empties all X -symbols via ε -transitions and returns to q with stack Z_0 , thereby resetting the depth to 0.

Determinism. In state q , every transition consumes an input symbol, and for each pair (input symbol, top-of-stack symbol) at most one transition is defined; no ε -transition is available. In state r , only the two ε -transitions are defined—one for top symbol X and one for Z_0 —and no input-consuming transition is available. Hence at every configuration there is at most one applicable move, so M is deterministic.

Correctness. For a word $w \in \{d, u, ;\}^*$ and a prefix p of w , let $\text{depth}(p)$ denote the counter value obtained by reading p left-to-right under the rules: d increments, u decrements, $;$ resets to 0—provided the value never goes negative. We establish the following invariant by induction on $|p|$.

Invariant. After reading any prefix p of the input and completing all subsequent ε -transitions, exactly one of the following holds:

- (i) the run has halted because some u was read while the stack contained only Z_0 ;
or
- (ii) M is in state q with stack $X^{\text{depth}(p)}Z_0$.

Base case. For $p = \varepsilon$, the machine starts in state q with stack Z_0 , matching case (ii) with $\text{depth}(\varepsilon) = 0$.

Inductive step. Assume the invariant holds for p , with case (ii) in force (otherwise there is nothing to prove). Let a be the next input symbol.

- If $a = d$: from state q , one X is pushed, yielding stack $X^{\text{depth}(p)+1}Z_0$ and state q . Since $\text{depth}(pa) = \text{depth}(p) + 1$, case (ii) holds.
- If $a = u$: if $\text{depth}(p) \geq 1$, the transition $\delta(q, u, X) = (q, \varepsilon)$ applies, yielding stack $X^{\text{depth}(p)-1}Z_0$ and state q , consistent with case (ii). If $\text{depth}(p) = 0$, the stack is Z_0 and $\delta(q, u, Z_0)$ is undefined, so the run halts; this is exactly case (i).
- If $a = ;$: the machine moves to state r (stack unchanged), then applies $\delta(r, \varepsilon, X) = (r, \varepsilon)$ repeatedly until all X -symbols are removed, and finally applies $\delta(r, \varepsilon, Z_0) = (q, Z_0)$. The result is state q with stack Z_0 , matching case (ii) since $\text{depth}(p;) = 0$.

This completes the induction.

($\supseteq$) Let $w \in \text{CCL}$. By definition, the depth never becomes negative during the left-to-right scan of w . Hence case (i) of the invariant never occurs, the entire run on w is defined, and after the last symbol and all ε -transitions, case (ii) places M in state $q \in F$. Thus $w \in L(M)$.

($\subseteq$) Let $w \notin \text{CCL}$. Then some block of w has a prefix p' with $\#_u(p') > \#_d(p')$. Consider the first position at which the depth would go negative; immediately before reading the corresponding u , the depth is 0. By the invariant, M is in state q with stack Z_0 . Since $\delta(q, u, Z_0)$ is undefined, the run halts before the input is exhausted, so $w \notin L(M)$.

We conclude $L(M) = \text{CCL}$. Since M is a DPDA, CCL is deterministic context-free. $\square$

Theorem 9.3. $\text{CCL} \notin \text{RS}$.

Proof. We use a pigeonhole argument. Suppose that CCL is $\sim_h$ -substitutable for some finite monoid homomorphism $h : \{d, u, ;\}^* \rightarrow M$. Since M is finite, there exist $1 \leq i < j$ such that $h(d^i) = h(d^j)$. Let $x := d^i$ and $y := d^j$.

We first show that $\mathcal{D}_{\text{CCL}}(x) \cap \mathcal{D}_{\text{CCL}}(y) \neq \emptyset$. Since $xu^i = d^i u^i \in \text{CCL}$ and $yu^i = d^j u^i \in \text{CCL}$, because in both words the running depth never becomes negative, we have $(\lambda, u^i) \in \mathcal{D}_{\text{CCL}}(x) \cap \mathcal{D}_{\text{CCL}}(y)$.

On the other hand, $yu^j = d^j u^j \in \text{CCL}$, but $xu^j = d^i u^j \notin \text{CCL}$. Indeed, after the first i occurrences of u the depth becomes 0, and the remaining $j - i > 0$ occurrences of u cause underflow. Hence $(\lambda, u^j) \in \mathcal{D}_{\text{CCL}}(y) \setminus \mathcal{D}_{\text{CCL}}(x)$, so $\mathcal{D}_{\text{CCL}}(x) \neq \mathcal{D}_{\text{CCL}}(y)$. This contradicts $\sim_h$ -substitutability. $\square$

Remark 9.4. The contrast with Section 8 is sharp: each capped language CCL_p belongs to RS, whereas its uncapped limit CCL does not, even though it is deterministic context-free.

9.2 The one-bracket Dyck language

Next we consider the standard one-bracket Dyck language over $\{a, b\}$, where a is an opening bracket and b is a closing bracket.

Definition 9.5 (The one-bracket Dyck language).

$$D_1 := \left\{ w \in \{a, b\}^* \mid \text{for every prefix } p \text{ of } w, \#(a, p) \geq \#(b, p), \text{ and } \#(a, w) = \#(b, w) \right\}.$$

Proposition 9.6. D_1 is deterministic context-free.

Proof. For completeness, we include a proof. Since a language L is a DCFL if and only if $L\$$ is a DCFL [14], it suffices to construct a DPDA for $\{w\$ \mid w \in D_1\}$, where $\$ \notin \{a, b\}$ is a right endmarker. Let $M = (Q, \{a, b, \$\}, \Gamma, \delta, q, Z_0, F)$ with $Q = \{q, q_f\}$, $\Gamma = \{Z_0, X\}$, and $F = \{q_f\}$. The transition function δ is defined by

$$\delta(q, a, Z_0) = (q, XZ_0), \quad \delta(q, a, X) = (q, XX), \quad \delta(q, b, X) = (q, \varepsilon), \quad \delta(q, \$, Z_0) = (q_f, Z_0),$$

with all other transitions undefined. Since every defined transition consumes an input symbol, there are no ε -moves, and for each triple (state, input symbol, top-of-stack symbol) at most one transition is defined, M is deterministic.

Invariant. We claim that if the run of M on a prefix p does not halt before p is exhausted, then upon finishing p the stack content is $X^{\#(a, p) - \#(b, p)} Z_0$. The proof is by induction on $|p|$. For $p = \varepsilon$ the stack is Z_0 and $\#(a, \varepsilon) - \#(b, \varepsilon) = 0$, so the base case holds. For the inductive step, assume the stack is $X^k Z_0$ where $k = \#(a, p) - \#(b, p)$.

- If the next symbol is a , then $\delta(q, a, \cdot)$ pushes one X regardless of the top symbol, so the stack becomes $X^{k+1} Z_0$, matching the updated difference $k + 1$.
- If the next symbol is b and $k \geq 1$, then the top is X and $\delta(q, b, X) = (q, \varepsilon)$ pops it, yielding stack $X^{k-1} Z_0$, matching $k - 1$.
- If the next symbol is b and $k = 0$, then the top is Z_0 and $\delta(q, b, Z_0)$ is undefined, so the run halts.

This completes the induction.

($\supseteq$) Let $w \in D_1$. Since every prefix p of w satisfies $\#(a, p) \geq \#(b, p)$, the third case above never arises while reading w , so the run does not halt prematurely. After all of w is read the stack is $X^{\#(a, w) - \#(b, w)} Z_0 = Z_0$, using $\#(a, w) = \#(b, w)$. Hence $\delta(q, \$, Z_0) = (q_f, Z_0)$ applies, and M accepts $w\$$.

($\subseteq$) Suppose M accepts $w\$$. The only transition on $\$$ is $\delta(q, \$, Z_0) = (q_f, Z_0)$, so immediately before reading $\$$ the machine must be in state q with top symbol Z_0 . By the invariant, $\#(a, w) - \#(b, w) = 0$, i.e. $\#(a, w) = \#(b, w)$. Moreover, since $\delta(q, b, Z_0)$ is undefined, the run cannot survive any prefix p with $\#(b, p) > \#(a, p)$; hence every prefix of w satisfies $\#(a, p) \geq \#(b, p)$. Therefore $w \in D_1$.

We conclude that $L(M) = \{w\$ \mid w \in D_1\}$, so D_1 is deterministic context-free. $\square$

Theorem 9.7. $D_1 \notin \text{RS}$.

Proof. Again we use a pigeonhole argument. Suppose that D_1 is $\sim_h$ -substitutable for some finite monoid homomorphism $h : \{a, b\}^* \rightarrow M$. Since M is finite, choose $1 \leq i < j$ such that $h(b^i a^i) = h(b^j a^j)$, and set $x := b^i a^i$ and $y := b^j a^j$.

We show $\mathcal{D}_{D_1}(x) \cap \mathcal{D}_{D_1}(y) \neq \emptyset$ using the common context (a^j, b^j) . Indeed, $a^j b^i a^i b^j \in D_1$, because the initial a^j creates depth j , then b^i lowers it to $j - i \geq 1$ (since $i < j$), the following a^i raises it back to j , and the final b^j returns it to 0. Similarly, $a^j b^j a^j b^j \in D_1$. Hence $(a^j, b^j) \in \mathcal{D}_{D_1}(x) \cap \mathcal{D}_{D_1}(y)$.

Now consider the context (a^i, b^i) . We have $a^i b^i a^i b^i \in D_1$, but $a^i b^j a^j b^i \notin D_1$, because the prefix $a^i b^j$ reaches depth $i - j < 0$. Hence $(a^i, b^i) \in \mathcal{D}_{D_1}(x) \setminus \mathcal{D}_{D_1}(y)$, so $\mathcal{D}_{D_1}(x) \neq \mathcal{D}_{D_1}(y)$. This contradicts $\sim_h$ -substitutability. $\square$

9.3 Yoshinaka's classical language

Finally, consider Yoshinaka's classical language $L_Y := L(S \rightarrow aSS \mid b)$.

Lemma 9.8. $L_Y = D_1 b$.

Proof. We show that $w \in L_Y$ iff $w = zb$ with $z \in D_1$.

We first record a structural fact about D_1 that follows directly from Definition 9.5: every $z \in D_1$ is either λ , or can be written as $z = az_1bz_2$ with $z_1, z_2 \in D_1$, obtained by splitting at the first position at which the running depth returns to 0.

($\subseteq$) We prove $L_Y \subseteq D_1 b$ by induction on the height of the derivation tree from S .

- If $S \Rightarrow b$, then $w = b = \lambda b$ and $\lambda \in D_1$.
- If $S \Rightarrow aSS \Rightarrow^* ax_1x_2$, then by the induction hypothesis $x_1 = z_1b$ and $x_2 = z_2b$ with $z_1, z_2 \in D_1$. Hence $w = az_1bz_2b$. The running depth of az_1bz_2 never goes negative (since $z_1, z_2 \in D_1$) and returns to 0 at the end, so $az_1bz_2 \in D_1$, giving $w \in D_1 b$.

($\supseteq$) We prove $D_1 b \subseteq L_Y$ by induction on $|z|$ for $z \in D_1$.

- If $z = \lambda$, then $zb = b$ and $S \Rightarrow b$.
- If $z = az_1bz_2$ with $z_1, z_2 \in D_1$, then by the induction hypothesis $z_1b, z_2b \in L_Y$, so $S \Rightarrow aSS \Rightarrow^* a(z_1b)(z_2b) = az_1bz_2b = zb$.

Hence $D_1 b \subseteq L_Y$. $\square$

Proposition 9.9. L_Y is deterministic context-free.

Proof. By Lemma 9.8, $L_Y = D_1 b$. Let $M = (Q, \{a, b, \$\}, \Gamma, \delta, q, Z_0, F)$ be the DPDA for $D_1 \$$ constructed in Proposition 9.6, which accepts $\{w\$ \mid w \in D_1\}$ by entering $q_f \in F$ on reading $\$$ from configuration (q, Z_0) . Define $M' = (Q, \{a, b\}, \Gamma, \delta', q, Z_0, F)$ by replacing the transition $\delta(q, \$, Z_0) = (q_f, Z_0)$ with

$$\delta'(q, b, Z_0) = (q_f, Z_0),$$

leaving all other transitions unchanged and keeping $F = \{q_f\}$. Then M' accepts $\{wb \mid w \in D_1\} = L_Y$, and determinism is preserved since the modified transition still consumes an input symbol and no conflict is introduced. Hence L_Y is deterministic context-free. $\square$

Theorem 9.10. $L_Y \notin \text{RS}$.

Proof. Suppose toward a contradiction that $L_Y \in \text{RS}$. By Lemma 9.8, we have $L_Y = D_1 b$, hence $D_1 = L_Y/b$. By Lemma 9.11 below, the class RS is closed under right quotient by a fixed word, so $D_1 \in \text{RS}$. This contradicts Theorem 9.7. Therefore $L_Y \notin \text{RS}$. $\square$

Lemma 9.11 (Closure under left and right quotient by a fixed word). *Let $L \subseteq \Sigma^*$ and let $r \in \Sigma^*$. Define the right quotient $L/r := \{w \in \Sigma^* \mid wr \in L\}$ and the left quotient $r \setminus L := \{w \in \Sigma^* \mid rw \in L\}$. If L is $\sim_h$ -substitutable for some finite monoid homomorphism $h : \Sigma^* \rightarrow M$, then L/r and $r \setminus L$ are also $\sim_h$ -substitutable with respect to the same h . Therefore RS is closed under left and right quotient by a fixed word.*

Proof. We prove the right quotient case; the left quotient is analogous.

Assume that L is $\sim_h$ -substitutable. Let $x, y \in \Sigma^*$ satisfy $h(x) = h(y)$ and $D_{L/r}(x) \cap D_{L/r}(y) \neq \emptyset$. Choose $(u, v) \in D_{L/r}(x) \cap D_{L/r}(y)$. Then $uxv, yv \in L/r$, hence $uxvr, uyvr \in L$. Therefore $(u, vr) \in D_L(x) \cap D_L(y)$. Since L is $\sim_h$ -substitutable and $h(x) = h(y)$, we obtain $D_L(x) = D_L(y)$.

Now let $(s, t) \in D_{L/r}(x)$ be arbitrary. Then $sxt \in L/r$, i.e. $sxtr \in L$. Hence $(s, tr) \in D_L(x) = D_L(y)$, so $sytr \in L$. Therefore $syt \in L/r$, i.e. $(s, t) \in D_{L/r}(y)$. Thus $D_{L/r}(x) \subseteq D_{L/r}(y)$. The reverse inclusion follows symmetrically, so $D_{L/r}(x) = D_{L/r}(y)$. Therefore L/r is $\sim_h$ -substitutable. $\square$

9.4 Consequence

The examples above show that recognizable-congruence control, although genuinely stronger than bounded prefix-suffix control, still misses important deterministic context-free benchmark languages.

Corollary 9.12. *The class RS does not contain all deterministic context-free languages.*

Proof. By Proposition 9.2 and Theorem 9.3, the deterministic context-free language CCL does not belong to RS. Likewise, combining Propositions 9.6 and 9.9 with Theorems 9.7 and 9.10, we obtain two further deterministic context-free languages lying outside RS. $\square$

Remark 9.13. *The overall picture is therefore clear. On the positive side, Section 8 provided regular witnesses for $\text{KL} \subsetneq \text{RS}$. On the negative side, the present section showed that even natural deterministic context-free languages such as CCL, D_1 , and L_Y lie outside RS.*

The boundary results above raised the question of whether the gap $\text{RS} \setminus \text{KL}$ contains any non-regular context-free language: the witnesses for $\text{KL} \subsetneq \text{RS}$ established in Section 8 are all regular, while every non-regular context-free benchmark considered in this section falls outside RS. Section 10 answers this question positively by showing that $L^* = \{a^n b^n : n \geq 0\}^*$ is a non-regular deterministic context-free language in $\text{RS} \setminus \text{KL}$, demonstrating that the extra expressive power of recognizable-congruence control over bounded prefix-suffix control persists strictly beyond the regular level.

10 A Non-Regular Context-Free Language in $\text{RS} \setminus \text{KL}$

We show that $L^* := \{a^n b^n : n \geq 0\}^*$ is a non-regular deterministic context-free language belonging to $\text{RS} \setminus \text{KL}$.

Theorem 10.1. *For the language $L^* = \{a^n b^n : n \geq 0\}^*$ over $\Sigma = \{a, b\}$, we have $L^* \in \text{RS} \setminus \text{KL}$. In particular, L^* is non-regular and deterministic context-free.*

For $w \in \{a, b\}^*$ and $0 \leq i \leq |w|$, define $\beta_w(i) := |w[1..i]|_a - |w[1..i]|_b$.

Lemma 10.2. *A word $w \in \{a, b\}^*$ belongs to L^* if and only if both of the following conditions hold.*

- (i) $\beta_w(i) \geq 0$ for all $0 \leq i \leq |w|$, and $\beta_w(|w|) = 0$.
- (ii) Whenever $w_i = b$ and $w_{i+1} = a$, we have $\beta_w(i) = 0$.

Proof. Let $w \in L^*$ and write $w = a^{n_1} b^{n_1} \dots a^{n_m} b^{n_m}$. Each block starts at height 0, never goes negative, and returns to height 0, so (i) holds. Every occurrence of ba is at a boundary between consecutive blocks, where the height is 0, so (ii) holds.

Conversely, assume (i) and (ii). Let $0 = i_0 < i_1 < \dots < i_m = |w|$ be the complete list of indices with $\beta_w(i_j) = 0$, and set $w^{(j)} := w[i_{j-1} + 1..i_j]$. By (i), each $w^{(j)}$ has positive relative height at every proper prefix and total relative height 0. If $w^{(j)}$ contained ba at some index r with $i_{j-1} < r < i_j$, then $\beta_w(r) > 0$, contradicting (ii). Hence $w^{(j)} \in a^* b^*$, and since its height is non-negative and total height is 0, we get $w^{(j)} = a^{n_j} b^{n_j}$ for some $n_j \geq 0$. Therefore $w = w^{(1)} \dots w^{(m)} \in L^*$. $\square$

Proposition 10.3. *L^* is non-regular and deterministic context-free.*

Proof. Since $L^* \cap a^* b^* = \{a^n b^n : n \geq 0\}$ is non-regular by the Pumping Lemma, L^* is non-regular.

For deterministic context-freeness, consider the DPDA M shown in Figure 3, with state set $Q = \{q_G, q_U, q_D\}$, input alphabet $\Sigma = \{a, b\}$, stack alphabet $\Gamma = \{Z_0, Y, X\}$, initial state q_G ,

initial stack symbol Z_0 , and accepting state set $F = \{q_G\}$. Every transition consumes an input symbol and there are no ε -transitions. Moreover, no transition is defined for any (state, input symbol, stack top) triple not shown in the figure; any computation reaching such a configuration is immediately rejected. Hence the machine is deterministic.

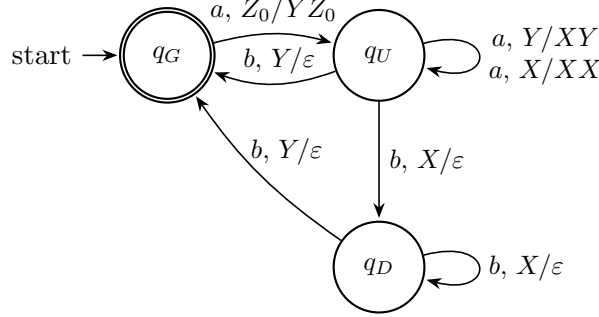


Figure 3: DPDA accepting $L^* = \{a^n b^n : n \geq 0\}^*$. The notation $\sigma, A/\gamma$ means “read input σ , pop A , push γ ”. State q_G is the inter-block boundary state, q_U reads the ascending part of the current block, and q_D reads the descending part. Transitions not shown are undefined; any computation reaching an undefined configuration is rejected.

The following invariant is proved by induction on the length of the input prefix. After reading prefix p (provided the machine has not yet rejected):

- (i) If the current state is q_G , then $p \in L^*$ and the stack is Z_0 .
- (ii) If the current state is q_U , then there exist $z \in L^*$ and $r \geq 1$ such that $p = za^r$ and the stack is $X^{r-1}YZ_0$.
- (iii) If the current state is q_D , then there exist $z \in L^*$, $n \geq 2$, and $1 \leq j < n$ such that $p = za^n b^j$ and the stack is $X^{n-j-1}YZ_0$.

Base case ($p = \lambda$). The initial state is q_G , the stack is Z_0 , and $\lambda \in L^*$, so (i) holds.

Inductive step. Assume the invariant holds for every prefix of length ℓ , and let σ be the next input symbol. There are seven defined transitions.

Transition T1: $(q_G, a, Z_0) \rightarrow (q_U, YZ_0)$. By the inductive hypothesis, the state is q_G , the stack is Z_0 , and $p \in L^*$. Reading $\sigma = a$ gives state q_U and stack $YZ_0 = X^0YZ_0$. Set $z := p \in L^*$ and $r := 1$; then $p' = pa = za$ and $X^{r-1}YZ_0 = X^0YZ_0$. Thus (ii) holds.

Transition T2: $(q_U, a, Y) \rightarrow (q_U, XY)$. By the inductive hypothesis, the state is q_U , the stack is $X^{r-1}YZ_0$ with stack top Y , i.e. $r = 1$, and $p = za$ ($z \in L^*$). Reading $\sigma = a$ gives state q_U and stack $XYZ_0 = X^1YZ_0$. Set $r' := 2$; then $p' = za^2$ and $X^{r'-1}YZ_0 = X^1YZ_0$. Thus (ii) holds.

Transition T3: $(q_U, a, X) \rightarrow (q_U, XX)$. By the inductive hypothesis, the state is q_U , the stack is $X^{r-1}YZ_0$ ($r \geq 2$), and $p = za^r$ ($z \in L^*$). Reading $\sigma = a$ gives state q_U and stack X^rYZ_0 . Set $r' := r + 1 \geq 2$; then $p' = za^{r+1}$ and $X^{r'-1}YZ_0 = X^rYZ_0$. Thus (ii) holds.

Transition T4: $(q_U, b, Y) \rightarrow (q_G, \varepsilon)$. By the inductive hypothesis, the state is q_U , the stack is $X^{r-1}YZ_0$ with stack top Y , i.e. $r = 1$, and $p = za$ ($z \in L^*$). Reading $\sigma = b$ gives state q_G and stack Z_0 . Then $p' = z(ab)$; since $z \in L^*$ and $ab = a^1b^1 \in L^*$, we have $p' \in L^*$. Thus (i) holds.

Transition T5: $(q_U, b, X) \rightarrow (q_D, \varepsilon)$. By the inductive hypothesis, the state is q_U , the stack is $X^{r-1}YZ_0$ ($r \geq 2$), and $p = za^r$ ($z \in L^*$). Reading $\sigma = b$ gives state q_D and stack $X^{r-2}YZ_0$. Set $n := r \geq 2$ and $j := 1$; then $1 \leq j < n$, $p' = za^r b$, and $X^{n-j-1}YZ_0 = X^{r-2}YZ_0$. Thus (iii) holds.

Transition T6: $(q_D, b, X) \rightarrow (q_D, \varepsilon)$. By the inductive hypothesis, the state is q_D , $p = za^n b^j$ ($n \geq 2$, $1 \leq j < n$), and the stack is $X^{n-j-1}YZ_0$ with stack top X , i.e. $n - j - 1 \geq 1$, i.e. $j < n - 1$. Reading $\sigma = b$ gives state q_D and stack $X^{n-j-2}YZ_0$. Set $j' := j + 1$; then $1 \leq j' < n$, $p' = za^n b^{j+1}$, and $X^{n-j'-1}YZ_0 = X^{n-j-2}YZ_0$. Thus (iii) holds.

Transition T7: $(q_D, b, Y) \rightarrow (q_G, \varepsilon)$. By the inductive hypothesis, the state is q_D , $p = za^n b^j$ ($n \geq 2$, $1 \leq j < n$), and the stack is $X^{n-j-1} Y Z_0$ with stack top Y , i.e. $n - j - 1 = 0$, i.e. $j = n - 1$. Reading $\sigma = b$ gives state q_G and stack Z_0 . Then $p' = za^n b^n$; since $z \in L^*$ and $a^n b^n \in L^*$, we have $p' \in L^*$. Thus (i) holds.

Deriving $L(M) = L^$.* The invariant shows that the machine reaches the accepting state q_G (without rejecting) after reading a complete input w if and only if $w \in L^*$. Hence $w \in L(M) \iff w \in L^*$, so $L(M) = L^*$ and L^* is deterministic context-free. $\square$

Proposition 10.4. $L^* \in \text{RS}$.

Proof. Set $L := L^*$. For $w \in \{a, b\}^*$, let $\text{fst}(w), \text{lst}(w) \in \{\perp, a, b\}$ denote the first and last symbols of w (with $\text{fst}(\lambda) = \text{lst}(\lambda) = \perp$), and let $\text{ba}(w) \in \{0, 1\}$ indicate whether w contains the factor ba . Set $M := \{\perp, a, b\} \times \{\perp, a, b\} \times \{0, 1\}$ and define $h : \{a, b\}^* \rightarrow M$ by

$$h(w) := (\text{fst}(w), \text{lst}(w), \text{ba}(w)).$$

We make M a monoid by declaring the product of (f, ℓ, c) and (f', ℓ', c') to be $(\tilde{f}, \tilde{\ell}, \tilde{c})$, where: $\tilde{f} := f$ if $f \neq \perp$, else $\tilde{f} := f'$; $\tilde{\ell} := \ell'$ if $\ell' \neq \perp$, else $\tilde{\ell} := \ell$; $\tilde{c} := c \vee c' \vee [\ell = b \wedge f' = a]$. The identity element is $(\perp, \perp, 0)$. We verify that h is a monoid homomorphism by checking $h(ww') = h(w) \cdot h(w')$ componentwise for all $w, w' \in \{a, b\}^*$.

First component (fst). The first symbol of ww' is the first symbol of w if $w \neq \lambda$, and the first symbol of w' if $w = \lambda$. By definition $\tilde{f} = f$ when $f \neq \perp$ (i.e. $w \neq \lambda$) and $\tilde{f} = f'$ when $f = \perp$ (i.e. $w = \lambda$), so $\tilde{f} = \text{fst}(ww')$.

Second component (lst). The last symbol of ww' is the last symbol of w' if $w' \neq \lambda$, and the last symbol of w if $w' = \lambda$. By definition $\tilde{\ell} = \ell'$ when $\ell' \neq \perp$ (i.e. $w' \neq \lambda$) and $\tilde{\ell} = \ell$ when $\ell' = \perp$ (i.e. $w' = \lambda$), so $\tilde{\ell} = \text{lst}(ww')$.

Third component (ba). The concatenation ww' contains the factor ba if and only if at least one of the following holds: (a) w contains ba internally; (b) w' contains ba internally; (c) the last symbol of w is b and the first symbol of w' is a . By definition $\tilde{c} = c \vee c' \vee [\ell = b \wedge f' = a]$ is exactly the disjunction of these three conditions, so $\tilde{c} = \text{ba}(ww')$.

Hence $h(ww') = (\tilde{f}, \tilde{\ell}, \tilde{c}) = h(w) \cdot h(w')$, confirming that h is a homomorphism into the finite monoid M .

We now show that L is $\sim_h$ -substitutable. Let $x, y \in \{a, b\}^*$ satisfy $h(x) = h(y)$ and $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset$. Pick $(u, v) \in \mathcal{D}_L(x) \cap \mathcal{D}_L(y)$, so $uxv, uyv \in L$. By symmetry it suffices to show $\mathcal{D}_L(x) \subseteq \mathcal{D}_L(y)$. Fix $(s, t) \in \mathcal{D}_L(x)$ with $sxt \in L$; we verify conditions (i) and (ii) of Lemma 10.2 for syt to conclude $syt \in L$.

Set $c := \beta_u(|u|)$, $k := \beta_s(|s|)$, and $d := \beta_x(|x|)$. Applying Lemma 10.2(i) to $uxv \in L$ gives $c \geq 0$, and to $sxt \in L$ gives $k \geq 0$. From $uxv, uyv \in L$ and $\beta_{uxv}(|uxv|) = \beta_{uyv}(|uyv|) = 0$ we obtain $\beta_y(|y|) = \beta_x(|x|) = d$.

Case 1: x contains ba . Since $h(x) = h(y)$, y also contains ba . Let i be any index with $x_i = b$ and $x_{i+1} = a$. Applying Lemma 10.2(ii) to $uxv \in L$ gives $c + \beta_x(i) = 0$, while applying it to $sxt \in L$ gives $k + \beta_x(i) = 0$. Since $c, k \geq 0$, we obtain $\beta_x(i) = -c = -k$, and hence $k = c$. Because i was arbitrary, every internal occurrence of ba in x has height $-c = -k$.

Condition (i) for syt . Heights of prefixes ending inside s are non-negative, same as in $sxt \in L$. The height at position j inside y is $k + \beta_y(j) = c + \beta_y(j) \geq 0$ (since $uyv \in L$). The height at prefix t' of t is $k + d + \beta_{t'}(|t'|)$, which is non-negative since the corresponding prefix of sxt is non-negative. Also $\beta_{syt}(|syt|) = k + d + \beta_t(|t|) = \beta_{sxt}(|sxt|) = 0$.

Condition (ii) for syt . Occurrences of ba internal to s or t have height 0 in $sxt \in L$. The occurrence of ba at the s - y boundary, if present, corresponds (via $\text{fst}(x) = \text{fst}(y)$) to the s - x boundary in sxt , where the height is $k = 0$. Similarly, the occurrence of ba at the y - t boundary corresponds (via $\text{lst}(x) = \text{lst}(y)$) to the x - t boundary in sxt , where the height is $k + d = 0$. For

an internal index j of y with $y_j = b$ and $y_{j+1} = a$, applying Lemma 10.2(ii) to $uyv \in L$ gives $c + \beta_y(j) = 0$; since $k = c$, the height in syt is $k + \beta_y(j) = 0$.

Hence $syt \in L$.

Case 2: x does not contain ba . Since $h(x) = h(y)$, y also does not contain ba , so $x, y \in a^*b^*$. For $z \in a^*b^*$, set $\mu(z) := \min_{0 \leq j \leq |z|} \beta_z(j)$. When $z \in a^*b^*$, this minimum depends only on $\beta_z(|z|)$ and $\text{fst}(z)$:

- If $z = \lambda$ or $z \in a^+$, then $\mu(z) = 0$.
- If $z \in b^+$, then $\mu(z) = \beta_z(|z|) \leq 0$.
- If $z \in a^+b^+$, then $\beta_z(j)$ attains its maximum $|z|_a$ at $j = |z|_a$ and decreases monotonically thereafter, so $\mu(z) = \beta_z(|z|)$.

In all cases $\mu(z) = \min(0, \beta_z(|z|))$. From $h(x) = h(y)$ and $\beta_x(|x|) = \beta_y(|y|) = d$ we get $\mu(x) = \mu(y)$.

Condition (i) for syt . Heights of prefixes ending inside s are unchanged. The height at position j inside y is $k + \beta_y(j) \geq k + \mu(y) = k + \mu(x) \geq 0$ (since $sxt \in L$). Prefixes extending into t , starting from height $k + d$ after completing y , are handled exactly as in Case 1. The total height is 0.

Condition (ii) for syt . There are no occurrences of ba internal to y . Occurrences of ba internal to s or t have height 0 in $sxt \in L$ (same argument as Case 1).

For the s - y boundary: ba appears there only when $\text{lst}(s) = b$ and $\text{fst}(y) = a$. Since $\text{fst}(x) = \text{fst}(y) = a$, the s - x boundary in sxt also carries ba . Applying Lemma 10.2(ii) to $sxt \in L$ at that boundary gives height $k = 0$, so the height at the s - y boundary in syt is also $k = 0$.

For the y - t boundary: ba appears there only when $\text{lst}(y) = b$ and $\text{fst}(t) = a$. Since $\text{lst}(x) = \text{lst}(y) = b$, the x - t boundary in sxt also carries ba . Applying Lemma 10.2(ii) to $sxt \in L$ at that boundary gives height $k + d = 0$, so the height at the y - t boundary in syt is also $k + d = 0$.

Hence $syt \in L$ in Case 2 as well.

In both cases we obtain $(s, t) \in \mathcal{D}_L(y)$. Therefore $\mathcal{D}_L(x) \subseteq \mathcal{D}_L(y)$, and by symmetry $\mathcal{D}_L(x) = \mathcal{D}_L(y)$. Hence L is $\sim_h$ -substitutable, so $L^* \in \text{RS}$. $\square$

Proposition 10.5. $L^* \notin \text{KL}$.

Proof. Since $\text{KL} = \bigcup_{k, \ell \geq 0} \{L \mid L \text{ is } (k, \ell)\text{-substitutable}\}$, it suffices to show that for every $k, \ell \geq 0$ the language L^* is not (k, ℓ) -substitutable. Fix $k, \ell \geq 0$ arbitrarily and choose $n > \max(k, \ell)$. Set $\xi := a^k$, $\eta := b^\ell$, $y_1 := a^{n-k}b^{n-\ell}$, $y_2 := a^{n-k}b^n a^n b^{n-\ell}$, $x_1 := \lambda$, $z_1 := \lambda$, $x_2 := a^n$, $z_2 := b^n$. Note $\xi y_1 \eta = a^n b^n \neq \lambda$ and $\xi y_2 \eta = a^n b^n a^n b^n \neq \lambda$.

The three words $x_1 \xi y_1 \eta z_1 = a^n b^n$, $x_1 \xi y_2 \eta z_1 = a^n b^n a^n b^n$, and $x_2 \xi y_1 \eta z_2 = a^{2n} b^{2n}$ all belong to L^* . On the other hand,

$$x_2 \xi y_2 \eta z_2 = a^{2n} b^n a^n b^{2n}$$

contains a central occurrence of ba immediately after the prefix $a^{2n} b^n$, at which point the height is $n > 0$. This violates Lemma 10.2(ii), so $x_2 \xi y_2 \eta z_2 \notin L^*$.

Hence L^* is not (k, ℓ) -substitutable for this fixed (k, ℓ) . Since $k, \ell \geq 0$ were arbitrary, L^* belongs to no (k, ℓ) -substitutable class, and therefore $L^* \notin \text{KL}$. $\square$

Proof of Theorem 10.1. Combine Propositions 10.3, 10.4, and 10.5. $\square$

11 Conclusion

We have established a finite typed reconstruction theory for distributional learning under a fixed recognizable congruence $\sim_h$ given by a finite monoid homomorphism $h : \Sigma^* \rightarrow M$. After passing to the typed refinement of a reduced grammar, the relevant syntactic information concentrates into finitely many typed local configurations exposed by a finite observation set $\text{CS}(\tilde{G})$, from which $\hat{G}(K)$ exactly reconstructs the target language. For every explicit h , the class $\mathcal{C}_h^{\text{cf}}$ is identifiable in the limit from positive data with polynomial-time hypothesis construction and

update; for the linear subclass $\mathcal{C}_h^{\text{lin}}$ we additionally obtain a complete polynomial time-and-data theorem.

We also situated the framework structurally. The capped counter family CCL_p witnesses $\text{KL} \subsetneq \text{RS}$ at the regular level, while CCL , D_1 , and $L_Y = L(S \rightarrow aSS \mid b)$ all lie outside RS . We further showed that $L^* = \{a^n b^n : n \geq 0\}^* \in \text{RS} \setminus \text{KL}$ is non-regular and deterministic context-free, establishing that $\text{KL} \subsetneq \text{RS}$ extends beyond the regular level.

The L^* result settles existence but not structure: a complete characterisation of $\text{RS} \cap \text{CFL}$ remains open and we regard it as the central problem left by this work. Beyond this, natural directions include the *unknown-h* setting in which the congruence must be inferred from data, and extensions to richer formalisms such as the MCFG settings [10, 11], reduction-based approaches [12], and substitutability over infinite alphabets [13].

Appendix A: Closure Properties and Counterexamples

In this appendix, we record several small structural facts concerning the classes studied in this paper.

A.1 Compatibility with inverse homomorphisms

Proposition A.1. *Let $\varphi : \Gamma^* \rightarrow \Sigma^*$ be a homomorphism. If $L \subseteq \Sigma^*$ is $\sim_h$ -substitutable, then $\varphi^{-1}(L)$ is $\sim_{h \circ \varphi}$ -substitutable.*

Proof. Assume that $(h \circ \varphi)(x) = (h \circ \varphi)(y)$ and $\mathcal{D}_{\varphi^{-1}(L)}(x) \cap \mathcal{D}_{\varphi^{-1}(L)}(y) \neq \emptyset$. Then there exists $(u, v) \in (\Gamma^*)^2$ such that $uxv, uyv \in \varphi^{-1}(L)$. Equivalently, $\varphi(u)\varphi(x)\varphi(v), \varphi(u)\varphi(y)\varphi(v) \in L$, so $\mathcal{D}_L(\varphi(x)) \cap \mathcal{D}_L(\varphi(y)) \neq \emptyset$. Since $h(\varphi(x)) = h(\varphi(y))$ and L is $\sim_h$ -substitutable, we obtain $\mathcal{D}_L(\varphi(x)) = \mathcal{D}_L(\varphi(y))$.

Now for arbitrary $(u, v) \in (\Gamma^*)^2$, we have $uxv \in \varphi^{-1}(L) \iff \varphi(u)\varphi(x)\varphi(v) \in L \iff \varphi(u)\varphi(y)\varphi(v) \in L \iff uyv \in \varphi^{-1}(L)$, and therefore $\mathcal{D}_{\varphi^{-1}(L)}(x) = \mathcal{D}_{\varphi^{-1}(L)}(y)$. $\square$

Corollary A.2. *The class RS is closed under inverse homomorphisms.*

Proof. Let $L \in \text{RS}$. Then there exists a finite monoid homomorphism $h : \Sigma^* \rightarrow M$ such that L is $\sim_h$ -substitutable. By Proposition A.1, for every $\varphi : \Gamma^* \rightarrow \Sigma^*$, the inverse image $\varphi^{-1}(L)$ is $\sim_{h \circ \varphi}$ -substitutable. Since $h \circ \varphi$ is again a finite monoid homomorphism, it follows that $\varphi^{-1}(L) \in \text{RS}$. $\square$

A.2 Compatibility with intersection

Proposition A.3. *The class RS is closed under finite intersection.*

Proof. Let $L_1, L_2 \in \text{RS}$. Then there exist finite monoid homomorphisms $h_1 : \Sigma^* \rightarrow M_1$ and $h_2 : \Sigma^* \rightarrow M_2$ such that L_1 is $\sim_{h_1}$ -substitutable and L_2 is $\sim_{h_2}$ -substitutable.

Define the product homomorphism $h : \Sigma^* \rightarrow M_1 \times M_2$ by $h(w) := (h_1(w), h_2(w))$. It suffices to show that $L_1 \cap L_2$ is $\sim_h$ -substitutable.

First, for every $z \in \Sigma^*$, we have $\mathcal{D}_{L_1 \cap L_2}(z) = \mathcal{D}_{L_1}(z) \cap \mathcal{D}_{L_2}(z)$. Indeed, $(u, v) \in \mathcal{D}_{L_1 \cap L_2}(z)$ is equivalent to $uzv \in L_1 \cap L_2$, which is equivalent to $uzv \in L_1$ and $uzv \in L_2$, and hence to $(u, v) \in \mathcal{D}_{L_1}(z) \cap \mathcal{D}_{L_2}(z)$.

Now let $x, y \in \Sigma^*$ satisfy $h(x) = h(y)$ and $\mathcal{D}_{L_1 \cap L_2}(x) \cap \mathcal{D}_{L_1 \cap L_2}(y) \neq \emptyset$. Then some (u, v) belongs to $\mathcal{D}_{L_1 \cap L_2}(x) \cap \mathcal{D}_{L_1 \cap L_2}(y)$, so by the above equality we have $(u, v) \in \mathcal{D}_{L_1}(x) \cap \mathcal{D}_{L_1}(y)$ and $(u, v) \in \mathcal{D}_{L_2}(x) \cap \mathcal{D}_{L_2}(y)$. Moreover, $h(x) = h(y)$ implies $h_1(x) = h_1(y)$ and $h_2(x) = h_2(y)$. Therefore, by the $\sim_{h_1}$ -substitutability of L_1 and the $\sim_{h_2}$ -substitutability of L_2 , we obtain $\mathcal{D}_{L_1}(x) = \mathcal{D}_{L_1}(y)$ and $\mathcal{D}_{L_2}(x) = \mathcal{D}_{L_2}(y)$. Hence $\mathcal{D}_{L_1 \cap L_2}(x) = \mathcal{D}_{L_1}(x) \cap \mathcal{D}_{L_2}(x) = \mathcal{D}_{L_1}(y) \cap \mathcal{D}_{L_2}(y) = \mathcal{D}_{L_1 \cap L_2}(y)$. Thus $L_1 \cap L_2$ is $\sim_h$ -substitutable.

Therefore $L_1 \cap L_2 \in \text{RS}$. $\square$

Corollary A.4. Let $L \subseteq \Sigma^*$ and let $R \subseteq \Sigma^*$ be regular. If $L \cap R \notin \text{RS}$, then $L \notin \text{RS}$.

Proof. Suppose toward a contradiction that $L \in \text{RS}$. Since every regular language belongs to RS , we have $R \in \text{RS}$. Therefore Proposition A.3 yields $L \cap R \in \text{RS}$, contradicting the assumption $L \cap R \notin \text{RS}$. Hence $L \notin \text{RS}$. $\square$

Remark A.5. The converse of Proposition A.3 does not hold in general. Indeed, it may happen that $L \notin \text{RS}$ but $L \cap R \in \text{RS}$ for some nonempty regular language R . For example, let $L := D_1$ and $R := \{ab\}$. By Theorem 9.7, we have $D_1 \notin \text{RS}$. On the other hand, $\{ab\}$ is finite, hence regular, and therefore belongs to RS . Moreover, since $ab \in D_1$, we have $D_1 \cap \{ab\} = \{ab\} \in \text{RS}$.

A.3 Counterexample: failure under Kleene star

Proposition A.6. There exists a language that is $\sim_h$ -substitutable for some homomorphism h , but whose Kleene closure is not $\sim_{h'}$ -substitutable for any finite monoid homomorphism h' .

Proof. Let $L := \{a^n b a^n \mid n \geq 0\}$. We first show that L is substitutable in the sense of Clark–Eyraud⁸. It is enough to prove that for all $x, y \in \Sigma^*$, if $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset$, then $\mathcal{D}_L(x) = \mathcal{D}_L(y)$. Take $(u, v) \in \mathcal{D}_L(x) \cap \mathcal{D}_L(y)$. Since $uxv, uyv \in L$, and every word in L contains exactly one occurrence of b , either both x and y contain b , or neither does.

Suppose first that neither x nor y contains b . Then we may write $x = a^r$ and $y = a^s$. Exactly one of u, v contains the unique occurrence of b . For example, if $u = u_1 b u_2$ and $v \in a^*$, then $uxv \in L$ is equivalent to $u_1, u_2, v \in a^*$ and $|u_1| = |u_2| + r + |v|$, while $uyv \in L$ is equivalent to $|u_1| = |u_2| + s + |v|$. Hence $r = s$, so $x = y$, and trivially $\mathcal{D}_L(x) = \mathcal{D}_L(y)$.

Suppose next that both x and y contain b . Then we may write $x = a^i b a^j$ and $y = a^{i'} b a^{j'}$. Since $uxv, uyv \in L$, we have $u, v \in a^*$. Writing $u = a^p$ and $v = a^q$, we get $a^{p+i} b a^{q+j}, a^{p+i'} b a^{q+j'} \in L$, hence $p + i = q + j$ and $p + i' = q + j'$, so $i - j = i' - j'$. On the other hand, for any $\alpha, \beta \geq 0$, the condition $(a^\alpha, a^\beta) \in \mathcal{D}_L(a^i b a^j)$ is equivalent to $\alpha + i = \beta + j$. Therefore $\mathcal{D}_L(a^i b a^j) = \{(a^\alpha, a^\beta) \mid \alpha + i = \beta + j\}$, and similarly $\mathcal{D}_L(a^{i'} b a^{j'}) = \{(a^\alpha, a^\beta) \mid \alpha + i' = \beta + j'\}$. Since $i - j = i' - j'$, these two sets are equal. Thus L is substitutable in the sense of Clark–Eyraud. In particular, by Proposition 3.4, L is $\sim_h$ -substitutable for some finite monoid homomorphism h .

We next show that L^* is not $\sim_{h'}$ -substitutable for any finite monoid homomorphism h' . Let M be an arbitrary finite monoid and let $h' : \Sigma^* \rightarrow M$ be an arbitrary homomorphism. By the pigeonhole principle, there exist $1 \leq k < \ell$ such that $h'(a^k) = h'(a^\ell)$. Put $x := b a^k b a^k b$ and $y := b a^\ell b a^\ell b$. Then $h'(x) = h'(b)h'(a^k)h'(b)h'(a^k)h'(b) = h'(b)h'(a^\ell)h'(b)h'(a^\ell)h'(b) = h'(y)$.

Moreover, (a^k, a^k) is a common context of x and y . Indeed, $a^k x a^k = a^k b a^k b a^k b a^k \in L^*$, corresponding to the factorization $(a^k b a^k)(b)(a^k b a^k)$, that is, to the parameters $n_1 = k$, $n_2 = 0$, and $n_3 = k$. Also, $a^k y a^k = a^k b a^\ell b a^\ell b a^k \in L^*$, corresponding to the factorization $(a^k b a^k)(a^{\ell-k} b a^{\ell-k})(a^k b a^k)$, that is, to the parameters $n_1 = k$, $n_2 = \ell - k$, and $n_3 = k$. Hence $(a^k, a^k) \in \mathcal{D}_{L^*}(x) \cap \mathcal{D}_{L^*}(y)$.

On the other hand, $(a^\ell, a^\ell) \in \mathcal{D}_{L^*}(y)$, since $a^\ell y a^\ell = a^\ell b a^\ell b a^\ell b a^\ell \in L^*$, corresponding to the parameters $n_1 = \ell$, $n_2 = 0$, and $n_3 = \ell$. However, $(a^\ell, a^\ell) \notin \mathcal{D}_{L^*}(x)$. Indeed, if $a^\ell x a^\ell = a^\ell b a^k b a^k b a^\ell$ belonged to L^* , then, because this word contains exactly three occurrences of b , there would exist $n_1, n_2, n_3 \geq 0$ such that $a^\ell b a^k b a^k b a^\ell = (a^{n_1} b a^{n_1})(a^{n_2} b a^{n_2})(a^{n_3} b a^{n_3})$. Expanding the right-hand side yields $a^{n_1} b a^{n_1+n_2} b a^{n_2+n_3} b a^{n_3}$, so comparing exponents gives $n_1 = \ell$, $n_1 + n_2 = k$, $n_2 + n_3 = k$, and $n_3 = \ell$. Hence $n_2 = k - \ell < 0$, a contradiction. Therefore $(a^\ell, a^\ell) \notin \mathcal{D}_{L^*}(x)$, and so $\mathcal{D}_{L^*}(x) \neq \mathcal{D}_{L^*}(y)$.

Thus we have found x, y such that $h'(x) = h'(y)$, $\mathcal{D}_{L^*}(x) \cap \mathcal{D}_{L^*}(y) \neq \emptyset$, and $\mathcal{D}_{L^*}(x) \neq \mathcal{D}_{L^*}(y)$. Therefore L^* is not $\sim_{h'}$ -substitutable. Since h' was arbitrary, L^* is not $\sim_{h'}$ -substitutable for any finite monoid homomorphism h' . $\square$

⁸Recall that Yoshinaka's $(0, 0)$ -substitutability can be written in the form: if $x, y \neq \lambda$ and $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset$, then $\mathcal{D}_L(x) = \mathcal{D}_L(y)$. Since the quantification is restricted to nonempty strings, this is weaker than Clark–Eyraud substitutability, which requires $\mathcal{D}_L(x) \cap \mathcal{D}_L(y) \neq \emptyset \Rightarrow \mathcal{D}_L(x) = \mathcal{D}_L(y)$ for all $x, y \in \Sigma^*$.

Appendix B: The Exceptional Status of CCL_1

In Section 8, we showed that for each fixed $p \geq 2$, the capped-counter language CCL_p is not (k, ℓ) -substitutable for any $k, \ell \geq 0$. The case $p = 1$ is exceptional. In this appendix, we show that CCL_1 belongs to Yoshinaka's hierarchy and lies exactly at the threshold $k, \ell \geq 1$.

Recall that $\text{CCL}_1 \subseteq \{d, u, ;\}^*$ consists of all words such that, in the left-to-right run where d changes the depth by $+1$, u changes it by -1 , and $;$ resets the depth to 0, the depth never drops below 0 and never exceeds 1.

Lemma B.1. *For $w \in \{d, u, ;\}^*$, the following are equivalent: $w \in \text{CCL}_1$; (i) w does not begin with u ; and (ii) w contains none of the factors dd , uu , and $;u$.*

Proof. Assume $w \in \text{CCL}_1$. Then the run cannot begin with u , since that would cause immediate underflow. The factor dd is impossible, because after the first d the depth is already 1. The factor uu is also impossible, because after the first u the depth is 0. Finally, $;u$ is impossible, because $;$ resets the depth to 0.

Conversely, assume that w does not begin with u and contains none of dd , uu , and $;u$. At depth 0, the next symbol must be either d or $;$, so the depth either becomes 1 or stays 0. At depth 1, the next symbol cannot be d , since dd is forbidden, and therefore it must be either u or $;$, both of which return the depth to 0. Hence the run always remains inside $\{0, 1\}$, so $w \in \text{CCL}_1$. $\square$

Lemma B.2. *If L is $(1, 1)$ -substitutable, then L is (k, ℓ) -substitutable for all $k, \ell \geq 1$.*

Proof. Fix $k, \ell \geq 1$, and assume that $|v| = k$, $|u| = \ell$, and $x_1vy_1uz_1, x_1vy_2uz_1, x_2vy_1uz_2 \in L$. Write $v = v'a$ and $u = bu'$. Let $x'_i := x_iv'$ and $z'_i := u'z_i$. Then the three premises become $x'_1ay_1bz'_1, x'_1ay_2bz'_1, x'_2ay_1bz'_2 \in L$. Applying $(1, 1)$ -substitutability, we obtain $x'_2ay_2bz'_2 = x_2vy_2uz_2 \in L$. $\square$

Proposition B.3. *CCL_1 is $(1, 1)$ -substitutable.*

Proof. Take $x_1vy_1uz_1, x_1vy_2uz_1, x_2vy_1uz_2 \in \text{CCL}_1$ with $v, u \in \Sigma$ and $vy_1u, vy_2u \neq \lambda$. We show $x_2vy_2uz_2 \in \text{CCL}_1$ using Lemma B.1.

The first symbol of $x_2vy_2uz_2$ coincides with the first symbol of $x_2vy_1uz_2$; if $x_2 \neq \lambda$, both begin with the first symbol of x_2 , and if $x_2 = \lambda$, both begin with v . Since $x_2vy_1uz_2 \in \text{CCL}_1$, this first symbol is not u .

Assume toward a contradiction that some forbidden factor $t \in \{dd, uu, ;u\}$ occurs in $x_2vy_2uz_2$.

- If it occurs inside x_2 or z_2 : then the same factor occurs in $x_2vy_1uz_2$, a contradiction.
- If it crosses the x_2 - v boundary or the u - z_2 boundary: then the same boundary letters also occur in $x_2vy_1uz_2$, again a contradiction.
- If it occurs inside vy_2u , or crosses one of its internal boundaries: then the same factor also occurs in $x_1vy_2uz_1$, a contradiction.

Every case leads to a contradiction, so $x_2vy_2uz_2 \in \text{CCL}_1$. $\square$

Corollary B.4. *If $k, \ell \geq 1$, then CCL_1 is (k, ℓ) -substitutable.*

Proof. This is immediate from Proposition B.3 and Lemma B.2. $\square$

Proposition B.5. *If $k = 0$ or $\ell = 0$, then CCL_1 is not (k, ℓ) -substitutable.*

Proof. To avoid confusion with the terminal symbol u of CCL_1 , we write the length-constraint blocks appearing in the definition of (k, ℓ) -substitutability as $p \in \Sigma^k$ and $q \in \Sigma^\ell$. In each case below we exhibit strings $x_1, y_1, z_1, x_2, y_2, z_2$ such that $x_1py_1qz_1, x_1py_2qz_1, x_2py_1qz_2 \in \text{CCL}_1$ yet $x_2py_2qz_2 \notin \text{CCL}_1$, thereby witnessing a failure of (k, ℓ) -substitutability. All membership and non-membership claims follow immediately from Lemma B.1.

Case 1: $(k, \ell) = (0, 0)$. Set $p = q = x_1 = z_1 = x_2 = \lambda$, $z_2 = u$, $y_1 = d$, and $y_2 = du$. Then $\lambda\lambda d\lambda\lambda = d$, $\lambda\lambda du\lambda\lambda = du$, and $\lambda\lambda d\lambda u = du$ all belong to CCL_1 , whereas $\lambda\lambda du\lambda u = duu$ contains the forbidden factor uu and hence does not.

Case 2: $k = 0$, $\ell \geq 1$. Set $p = \lambda$, $q = ;^\ell$, $x_1 = d$, $z_1 = x_2 = z_2 = \lambda$, $y_1 = \lambda$, and $y_2 = u$. Then $d\lambda\lambda q\lambda = dq = d;^\ell$, $d\lambda uq\lambda = du;^\ell$, and $\lambda\lambda\lambda q\lambda = q = ;^\ell$ all belong to CCL_1 , whereas $\lambda\lambda uq\lambda = uq = u;^\ell$ begins with u and hence does not.

Case 3: $k \geq 1$, $\ell = 0$. Set $p = ;^k$, $q = \lambda$, $x_1 = z_1 = x_2 = \lambda$, $z_2 = u$, $y_1 = d$, and $y_2 = \lambda$. Then $\lambda p d\lambda\lambda = pd = ;^k d$, $\lambda p\lambda\lambda\lambda = p = ;^k$, and $\lambda p d\lambda u = pdu = ;^k du$ all belong to CCL_1 , whereas $\lambda p\lambda\lambda u = pu = ;^k u$ contains the forbidden factor $;u$ and hence does not.

In all three cases (k, ℓ) -substitutability fails, so CCL_1 is not (k, ℓ) -substitutable whenever $k = 0$ or $\ell = 0$. $\square$

Proposition B.6. CCL_1 is (k, ℓ) -substitutable if and only if $k, \ell \geq 1$.

Proof. This is immediate from Proposition B.5 and Corollary B.4. $\square$

Remark B.7. Thus CCL_1 is exceptional within the capped-counter family: for each $p \geq 2$, Theorem 8.6 shows that CCL_p lies outside the whole Yoshinaka hierarchy, whereas CCL_1 already belongs to the smallest nontrivial level $(1, 1)$, and hence to every level (k, ℓ) with $k, \ell \geq 1$.

Data availability

No data were used for the research described in this article.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have influenced the work reported in this paper.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Acknowledgments

The author thanks Professor Makoto Kanazawa (Hosei University) for longstanding guidance, encouragement, and valuable comments on this research. Any remaining errors are solely the author's responsibility.

References

- [1] M. Kanazawa. *Learnable Classes of Categorical Grammars*. CSLI Publications, Stanford, CA, 1998.
- [2] E. M. Gold. Language identification in the limit. *Information and Control*, 10(5):447–474, 1967.
- [3] S. Eilenberg. *Automata, Languages, and Machines*, volume A. Academic Press, 1974.
- [4] C. de la Higuera. *Grammatical Inference: Learning Automata and Grammars*. Cambridge University Press, 2010.

- [5] C. de la Higuera. Characteristic sets for polynomial grammatical inference. *Machine Learning*, 27:125–138, 1997.
- [6] J. Heinz, C. de la Higuera, and M. van Zaanen. Formal and empirical grammatical inference. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Tutorial Abstracts*, pages 1–83, 2011.
- [7] D. Angluin. Inference of reversible languages. *Journal of the ACM*, 29(3):741–765, 1982.
- [8] A. Clark and R. Eyraud. Polynomial identification in the limit of substitutable context-free languages. *Journal of Machine Learning Research*, 8:1725–1745, 2007.
- [9] R. Yoshinaka. Identification in the limit of k, l -substitutable context-free languages. In *Proceedings of ICGI*, 2008.
- [10] R. Yoshinaka. Efficient learning of multiple context-free languages with multidimensional substitutability from positive data. *Theoretical Computer Science*, 412(19):1821–1831, 2011. doi:10.1016/j.tcs.2010.12.058.
- [11] A. Clark and R. Yoshinaka. Distributional learning of parallel multiple context-free grammars. *Machine Learning*, 96(1–2):5–31, 2014.
- [12] F. Coste and J. Nicolas. Learning local substitutable context-free languages from positive examples in polynomial time and data by reduction. In *Proceedings of the 14th International Conference on Grammatical Inference 2018, Proceedings of Machine Learning Research*, 93:155–168, 2019.
- [13] Y. Numaya, D. Hendrian, R. Yoshinaka, and A. Shinohara. Identification of substitutable context-free languages over infinite alphabets from positive data. In *Proceedings of the 16th International Conference on Grammatical Inference, Proceedings of Machine Learning Research*, 217:23–34, 2023.
- [14] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Addison-Wesley, 2006.

Declaration of Competing Interests

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.